

INFORMATIQUE • TLEE

MODULE 3 : ALGORITHMIQUE ET PROGRAMMATION

## LEÇON N08

PROGRAMMATION EN LANGAGE  
C

DURÉE CONSEILLÉE : 8 H

## DANS CETTE LEÇON

Cette leçon fait passer l'élève de l'algorithmique à la programmation réelle en langage C : structure d'un programme, entrées/sorties, structures de contrôle, tableaux et enregistrements. Elle culmine avec la traduction et l'exécution complète de l'algorithme de recherche séquentielle, compétence directement exigible au baccalauréat.

- À la fin de cette leçon, je sais distinguer langage de programmation, compilateur et environnement de développement (IDE) comme Dev-C++ et Code ::Blocks
- À la fin de cette leçon, je sais écrire la structure complète d'un programme C : bibliothèques stdio.h et math.h, fonction main, déclarations
- À la fin de cette leçon, je sais déclarer des variables et des constantes en C avec les types appropriés (int, float, char, const)
- À la fin de cette leçon, je sais traduire Lire et Écrire par scanf() et printf() avec les bons formats (%d, %f, %c)
- À la fin de cette leçon, je sais traduire les structures de contrôle d'un algorithme en C : if, for, while, do...while
- À la fin de cette leçon, je sais déclarer et manipuler un tableau en C (saisie, affichage, parcours)

TÉLÉCHARGE  
ONBUCH

Gratuit • Google Play  
Tuteur IA, annales,  
concours, résultats



## PASSE À ONBUCH+

Tous les cours signés, Tuteur IA  
illimité, fiches PDF — onglet  
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

## COURS

complet, illustré, pas à  
pas

## MÉTHODES

et exercices résolus

## EXERCICES

type examen + corrigés

## FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.  
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

**SOMMAIRE**

1. Langages, compilation et environnements de développement
2. Structure d'un programme C et déclarations
3. Entrées et sorties : scanf() et printf()
4. Structures de contrôle en C
5. Tableaux et enregistrements (struct) en C
6. Étude de cas complète : la recherche séquentielle en C
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

**OBJECTIFS DE LA LEÇON**

- À la fin de cette leçon, je sais distinguer langage de programmation, compilateur et environnement de développement (IDE) comme Dev-C++ et Code ::Blocks
- À la fin de cette leçon, je sais écrire la structure complète d'un programme C : bibliothèques stdio.h et math.h, fonction main, déclarations
- À la fin de cette leçon, je sais déclarer des variables et des constantes en C avec les types appropriés (int, float, char, const)
- À la fin de cette leçon, je sais traduire Lire et Écrire par scanf() et printf() avec les bons formats (%d, %f, %c)
- À la fin de cette leçon, je sais traduire les structures de contrôle d'un algorithme en C : if, for, while, do...while
- À la fin de cette leçon, je sais déclarer et manipuler un tableau en C (saisie, affichage, parcours)
- À la fin de cette leçon, je sais déclarer et utiliser un enregistrement avec struct en C
- À la fin de cette leçon, je sais traduire, compiler et exécuter en C l'algorithme de recherche séquentielle dans un tableau

**PRÉREQUIS**

- Notions d'algorithme, de variable et de type de données (Première)
- Structures algorithmiques : alternative (Si...Alors...Sinon) et répétitives (Pour,

TantQue, Répéter)

- Tableaux et enregistrements en algorithmique (Première)
- Algorithme de recherche séquentielle d'un élément dans un tableau
- Notion de fonction et de paramètre en algorithmique

# 1 LANGAGES, COMPILATION ET ENVIRONNEMENTS DE DEVELOPPEMENT

La coopérative de quartier de Mballa II à Yaoundé tient encore ses comptes sur un cahier : calcul des cotisations, recherche d'un membre dans la liste, bilan mensuel, tout se fait à la main et les erreurs s'accumulent. Le trésorier, qui sait que son fils écrit des algorithmes au lycée, demande : « Peut-on transformer ces algorithmes en vrais programmes qui tournent sur l'ordinateur du cybercafé ? » C'est exactement le rôle d'un **langage de programmation** comme le C : traduire un algorithme en un texte que la machine peut compiler puis exécuter. Dans cette section, nous répondons à la problématique suivante : **comment passe-t-on concrètement de l'algorithme, écrit en français, au programme C exécuté par l'ordinateur ?**

## 1.1 De l'algorithme au programme

### Rappel : algorithme et programme

Au chapitre précédent, tu as appris à écrire des algorithmes en pseudo-code : une suite finie et ordonnée d'instructions (Lire, Écrire, Si... Alors, Pour, Tant que) qui résout un problème. Un point fondamental doit être bien compris :

#### À RETENIR Algorithme et programme

Un **algorithme** est indépendant de tout langage : il décrit la méthode de résolution en français structuré. Un **programme** est la traduction de cet algorithme dans un langage de programmation précis (C, Pascal, Python...), de manière à pouvoir être exécuté par l'ordinateur. Le même algorithme peut donc donner naissance à plusieurs programmes, écrits dans des langages différents.

Reprenons l'exemple du trésorier : l'algorithme « calculer la cotisation totale d'un membre » s'écrit en pseudo-code :

```
Algorithme Cotisation
Variable montant, penalite, total : Reel
Debut
    Ecrire("Entrer le montant : ")
    Lire(montant)
    Ecrire("Entrer la penalite : ")
    Lire(penalite)
    total <- montant + penalite
    Ecrire("Total a payer : ", total)
Fin
```

Ce texte est parfaitement clair pour un être humain, mais l'ordinateur ne comprend ni « Lire » ni « Écrire ». Il faut le traduire dans un langage que la machine saura traiter : c'est le travail du programmeur, et le résultat s'appelle le **code source**.

```
#include <stdio.h>

int main()
```

```
{  
    float montant, penalite, total;  
    printf("Entrer le montant : ");  
    scanf("%f", &montant);  
    printf("Entrer la penalite : ");  
    scanf("%f", &penalite);  
    total = montant + penalite;  
    printf("Total a payer : %f", total);  
    return 0;  
}
```

## 1.2 Les langages de programmation

### DÉFINITION Langage de programmation

Un **langage de programmation** est un langage artificiel doté d'un vocabulaire limité et d'une syntaxe rigoureuse, permettant d'écrire des programmes exécutables par l'ordinateur. Contrairement au français, il ne tolère aucune ambiguïté ni aucune faute : un point-virgule oublié suffit à rendre le texte incompréhensible pour la machine.

Il existe des centaines de langages de programmation. En voici quelques-uns, avec leur domaine de prédilection :

| Langage | Caractéristiques                            | Usages typiques                              |
|---------|---------------------------------------------|----------------------------------------------|
| C       | Compilé, rapide, proche de la machine       | Systèmes d'exploitation, logiciels embarqués |
| Pascal  | Compilé, très pédagogique                   | Enseignement de la programmation             |
| Python  | Interprété, syntaxe simple                  | Sciences des données, scripts, enseignement  |
| Java    | Compilé puis interprété (machine virtuelle) | Applications d'entreprise, Android           |

### PROPRIÉTÉ Le langage C

Le langage **C** est un langage :

- compilé : le code source est traduit une fois pour toutes en langage machine (fichier exécutable) avant l'exécution ;
- proche de la machine : il permet de gérer finement la mémoire, ce qui le rend très rapide ;
- très utilisé : c'est le langage des systèmes d'exploitation (Windows, Linux), des logiciels embarqués et des applications exigeant de la performance.

C'est pour ces raisons que le programme officiel de Terminale le retient comme langage d'apprentissage.

### LESAVAIS-TU ? Le langage C, un cinquantenaire toujours jeune

Le langage C a été créé en 1972 par Dennis Ritchie dans les laboratoires Bell (États-Unis), pour réécrire le système d'exploitation UNIX. Plus de cinquante ans plus tard, il sert encore à écrire les systèmes d'exploitation modernes (le cœur de Linux, de Windows), les logiciels embarqués des téléphones, des routeurs et même des cartes électroniques utilisées dans les équipements télécoms déployés le long de la fibre optique nationale du Cameroun. Apprendre le C, c'est apprendre la langue dans laquelle une grande partie du monde numérique est écrite.

## 1.3 La chaîne de production d'un programme C : la compilation

L'ordinateur ne comprend qu'un seul langage : le **langage machine**, fait de suites de 0 et de 1. Or le code source C est écrit en anglais structuré (if, while, printf...). Il faut donc un traducteur : c'est le **compilateur**.

### DÉFINITION Compilateur

Le **compilateur** est un logiciel qui traduit le code source (fichier d'extension `.c`) en langage machine, produisant un fichier exécutable (sous Windows, extension `.exe`). Au passage, il vérifie la syntaxe du programme et signale les erreurs : tant qu'il reste une erreur, aucun exécutable n'est produit.

La chaîne complète de production d'un programme C est donc la suivante :



Chaîne de production d'un programme C : de l'algorithme aux résultats.

### ATTENTION Erreur de compilation ou résultat faux ?

Il faut distinguer deux types d'erreurs, très souvent confondus au Bac :

- l'**erreur de compilation** (erreur de syntaxe) : le programme ne respecte pas la grammaire du C (point-virgule manquant, parenthèse non fermée, variable non déclarée). Le compilateur la signale avec le numéro de la ligne et refuse de produire l'exécutable ;
- l'**erreur d'exécution** (erreur de logique) : le programme compile parfaitement, s'exécute... mais donne un résultat faux, parce que l'algorithme traduit est incorrect (par

exemple `total = montant - penalite;` au lieu de `+`). Le compilateur ne peut pas la détecter : seuls des tests avec des données connues la révèlent.

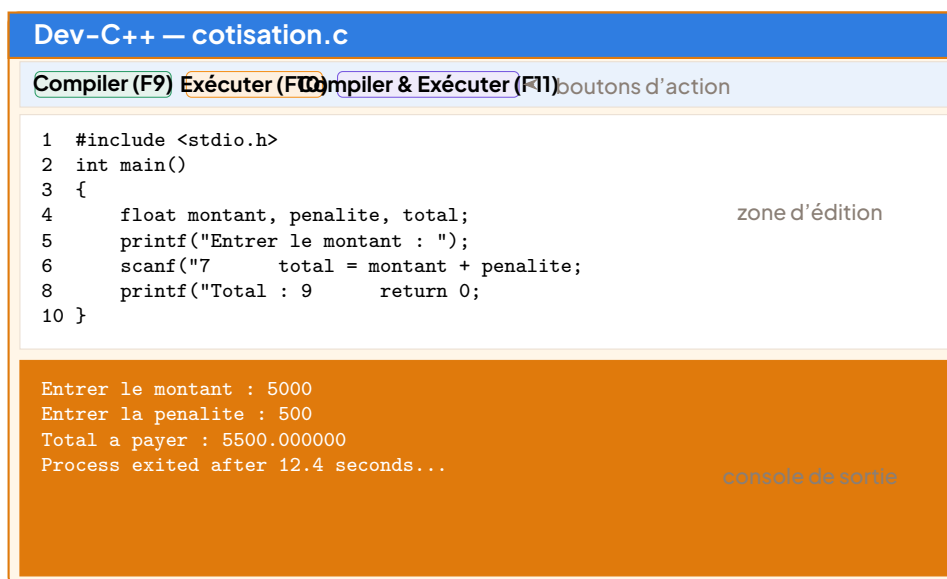
Un programme qui compile n'est donc pas forcément un programme juste !

## 1.4 Les environnements de développement intégrés (IDE)

Pour travailler, le programmeur a besoin de plusieurs outils : un éditeur pour taper le code, un compilateur pour le traduire, un moyen d'exécuter le programme et de repérer les erreurs. Plutôt que d'utiliser ces outils séparément, on les regroupe dans un seul logiciel.

### DÉFINITION IDE

Un **environnement de développement intégré** (IDE, de l'anglais Integrated Development Environment) est un logiciel qui regroupe dans une même fenêtre : un éditeur de code (avec coloration syntaxique et numérotation des lignes), un compilateur, des outils d'exécution et de débogage (recherche d'erreurs). Au lycée, nous utilisons Dev-C++ ou Code ::Blocks.



Fenêtre stylisée d'un IDE : éditeur de code, boutons Compiler/Exécuter, console de sortie.

### PROPRIÉTÉ Dev-C++ et Code ::Blocks

Dev-C++ et Code ::Blocks sont deux IDE libres et gratuits, adaptés à l'apprentissage du

langage C. Dans les deux cas :

- on crée un fichier source (menu Fichier → Nouveau → Fichier source) que l'on enregistre avec l'extension `.c` ;
- on compile avec la touche F9 (Dev-C++) ou en cliquant sur l'icône Construire (Code ::Blocks) : les erreurs éventuelles s'affichent en bas de la fenêtre avec leur numéro de ligne ;
- on exécute avec F10 (Dev-C++) ou l'icône Exécuter ; la combinaison F11 sous Dev-C++ compile puis exécute en une seule étape ;
- un double-clic sur un message d'erreur positionne le curseur sur la ligne fautive.

#### MÉTHODE Le cycle de travail en salle informatique

Pour transformer un algorithme en programme qui fonctionne, adopte toujours le cycle suivant :

1. Écrire le code source dans l'éditeur de l'IDE, en traduisant fidèlement l'algorithme ;
2. Enregistrer le fichier avec l'extension `.c` (par exemple `cotisation.c`) ;
3. Compiler (F9 ou F11) : si des erreurs sont signalées, lire le premier message, corriger, puis recompiler — toujours commencer par la première erreur, les suivantes en découlent souvent ;
4. Exécuter le programme obtenu ;
5. Tester avec des données connues dont on a calculé le résultat à la main (ex. montant = 5000, pénalité = 500 ⇒ total attendu : 5500). Si le résultat est faux, c'est une erreur de logique : relire l'algorithme, corriger le code et recommencer au point 3.

#### EXPÉRIENCE Premier programme : compiler et exécuter sous Dev-C++

Objectif : découvrir la chaîne de production complète sur un exemple simple.

Matériel : un poste informatique équipé de Dev-C++ (ou Code ::Blocks).

1. Lance Dev-C++, crée un nouveau fichier source et enregistre-le sous le nom `bonjour.c`.
2. Recopie exactement le code suivant :

```
#include <stdio.h> /* bibliotheque des entrees/sorties */  
  
int main()  
{  
    printf("Bonjour la cooperative de Mballa II !\n");  
}
```

```
    return 0;    /* le programme s'est bien termine */  
}
```

3. Appuie sur F11 (Compiler & Exécuter). Observation : une console noire s'ouvre et affiche le message.
4. Supprime maintenant le point-virgule de la ligne `printf` et recompile. Observation : le compilateur signale une erreur de syntaxe (expected ';') avec le numéro de ligne, et aucun exécutable n'est produit.
5. Corrige, recompile, réexécute. Interprétation : tu viens de vivre le cycle complet écrire → compiler → corriger → exécuter, et de distinguer erreur de syntaxe et exécution normale.

#### EXERCICE RÉSOLU 1 Identifier le type d'erreur

Énoncé. Trois élèves écrivent un programme C pour calculer la moyenne de deux notes  $a$  et  $b$ . Indique, pour chaque cas, s'il s'agit d'une erreur de compilation ou d'une erreur de logique, et justifie.

1. Élève 1 : il écrit `float moyenne` sans point-virgule à la fin de la ligne.
2. Élève 2 : il écrit `moyenne = a + b / 2;` et obtient un résultat inattendu.
3. Élève 3 : il écrit `printf("Moyenne : %f", moyenne);` et le programme affiche bien une valeur correcte.

#### SOLUTION

Solution détaillée.

1. Erreur de compilation : le point-virgule manquant est une faute de syntaxe ; le compilateur la signale et refuse de produire l'exécutable.
2. Erreur de logique : la syntaxe est correcte, donc le programme compile. Mais la division est prioritaire sur l'addition : l'expression calcule  $a + \frac{b}{2}$  au lieu de  $\frac{a+b}{2}$ . Il fallait écrire `moyenne = (a + b) / 2;`. Seul un test avec des valeurs connues (ex.  $a = 10$ ,  $b = 14$ , moyenne attendue 12) permet de détecter cette erreur.
3. Aucune erreur : la syntaxe est correcte et le résultat affiché est juste ; le programme remplit sa fonction.

#### EXERCICE 1 Pour t'entraîner

1. Définis les termes suivants : langage de programmation, compilateur, IDE.

2. Replace dans l'ordre les étapes de la chaîne de production : exécutable, algorithme, compilation, résultats, code source.
3. Le trésorier de la coopérative écrit un programme qui compile sans erreur mais affiche un bilan mensuel négatif alors que la caisse est excédentaire. De quel type d'erreur s'agit-il ? Quelle démarche doit-il suivre pour la corriger ?

**CORRIGÉ Exercice 1**

1. Langage de programmation : langage artificiel doté d'une syntaxe rigoureuse permettant d'écrire des programmes exécutables par l'ordinateur. Compilateur : logiciel qui traduit le code source en langage machine (exécutable) en vérifiant la syntaxe. IDE : logiciel regroupant éditeur de code, compilateur et outils d'exécution/débogage (ex. Dev-C++, Code ::Blocks).
2. Algorithme → code source → compilation → exécutable → résultats.
3. Il s'agit d'une erreur de logique (d'exécution) : le programme compile, donc la syntaxe est correcte, mais l'algorithme traduit est faux (par exemple une soustraction au lieu d'une addition). Il doit tester le programme avec des données dont il connaît le résultat calculé à la main, comparer, repérer l'instruction fautive, corriger le code, recompiler et retester.

**À RETENIR L'essentiel de la section**

- Un algorithme est indépendant de tout langage ; un programme en est la traduction dans un langage précis comme le C.
- Le C est un langage compilé : le code source (.c) est traduit par le compilateur en un fichier exécutable (.exe).
- Un IDE (Dev-C++, Code ::Blocks) regroupe éditeur, compilateur et outils d'exécution ; on compile avec F9/F11 et on lit les messages d'erreur.
- Erreur de syntaxe ⇒ pas de compilation ; erreur de logique ⇒ compilation réussie mais résultat faux : seuls les tests la détectent.

Dans la section suivante, nous ouvrirons le capot du programme C : structure générale, bibliothèques `stdio.h` et `math.h`, fonction `main` et déclarations de variables et de constantes.

## 2 STRUCTURE D'UN PROGRAMME C ET DÉCLARATIONS

Dans la section précédente, nous avons vu qu'un programme écrit en langage C est un simple fichier texte (le **code source**) qu'un **compilateur** traduit en langage machine avant exécution, et que des environnements de développement comme Dev-C++ ou Code::Blocks nous offrent un éditeur, un compilateur et un débogueur réunis dans une même fenêtre. Il reste une question essentielle : comment ce fichier texte doit-il être organisé pour être accepté par le compilateur ? Le langage C, comme toute langue, possède une grammaire stricte. Cette section te donne la « carte d'identité » de tout programme C : sa structure générale, ses bibliothèques, sa fonction principale, et la manière d'y déclarer variables et constantes. C'est le squelette sur lequel tout le reste du module va se greffer.

### 2.1 Structure générale d'un programme C

**Intuition.** Imagine une lettre administrative camerounaise : elle commence toujours par un en-tête (le ministère, la référence), puis vient le corps du message, puis la signature. Un programme C obéit à la même discipline : il commence par des **directives** qui annoncent les outils dont le programme aura besoin, puis vient une fonction principale qui contient le travail à accomplir, et il se termine par un signal indiquant que tout s'est bien passé.

#### DÉFINITION Programme C

Un **programme C** est un fichier texte (d'extension `.c`) composé, dans l'ordre, de :

1. des **directives du préprocesseur** (lignes commençant par `#`), qui demandent l'inclusion de bibliothèques ;
2. éventuellement des **déclarations globales** (constantes ou variables visibles par tout le programme) ;
3. la **fonction principale** `main()`, qui contient les instructions à exécuter, délimitées par des accolades `{ }` et terminée par `return 0;`.

Voici le plus petit programme C correct, celui que tout élève doit savoir écrire les yeux fermés :

```
#include <stdio.h> /* inclusion de la bibliotheque d'entrees/sorties */

int main()          /* fonction principale : point d'entree du programme */
{
    /* declarations des variables */

    /* instructions du programme */

    return 0;        /* le programme signale qu'il s'est termine normalement */
}
```

```
#include <stdio.h>
```

```
int main() {
```

```
float n1, n2, m;
```

```
scanf("%f", &n1);  
m = (n1 + n2) / 2;
```

```
return 0;  
}
```

**Directives** : on annonce les bibliothèques utilisées (ici les entrées/sorties).

**Fonction main()** : point d'entrée obligatoire ; l'accolade ouvre le corps du programme.

**Déclarations** : on réserve les cases mémoire (variables) avant

**Instructions** : le travail du programme (saisies, calculs, affichages). Chaque instruction se termine par « ; ».

**return 0 ;** : fin normale du programme ; l'accolade ferme le corps de main.

Anatomie d'un programme C minimal : chaque zone a un rôle précis et une place imposée.

#### MÉTHODE Le squelette type à reproduire

Pour écrire n'importe quel programme C de ce cours, suis toujours la même démarche :

1. Écris en première ligne `#include <stdio.h>` (et `#include <math.h>` si tu fais des calculs comme des racines carrées ou des puissances).
2. Écris `int main()` suivi d'une accolade ouvrante `{`.
3. Déclare toutes tes variables et constantes en début de corps, avant toute instruction.
4. Écris les instructions (saisies, calculs, affichages), chacune terminée par un point-virgule.
5. Termine par `return 0 ;` puis l'accolade fermante `}`.
6. Enregistre le fichier avec l'extension `.c`, compile, corrige les erreurs signalées, puis exécute.

## 2.2 Les bibliothèques : `stdio.h` et `math.h`

**Intuition.** Un menuisier ne fabrique pas ses clous : il les achète tout faits. De même, le programmeur C ne réécrit pas les opérations courantes (afficher à l'écran, lire au clavier, calculer une racine carrée) : il utilise des **bibliothèques**, c'est-à-dire des collections de fonctions déjà écrites, testées et prêtes à l'emploi.

#### DÉFINITION Bibliothèque

Une **bibliothèque** (ou fichier d'en-tête, d'extension `.h`) est un fichier fourni avec le lan-

gage C qui contient des **fonctions prêtes à l'emploi** (plus précisément leurs prototypes). On l'intègre au programme par la directive `#include <nom.h>` placée en tout début de fichier. Les deux bibliothèques au programme sont :

- `stdio.h` (standard input/output) : les entrées/sorties, avec les fonctions `printf()` (afficher) et `scanf()` (saisir);
- `math.h` : les fonctions mathématiques, comme `sqrt(x)` (racine carrée) et `pow(x, n)` (puissance  $x^n$ ).

#### PROPRIÉTÉ Syntaxe de la directive d'inclusion

La directive d'inclusion s'écrit exactement :

```
#include <nom_de_la_bibliotheque.h>
```

Elle commence par le symbole #, ne contient pas de point-virgule final, et le nom du fichier est encadré par les chevrons < et >. Elle est traitée avant la compilation proprement dite (c'est une directive du **préprocesseur**).

#### EXEMPLE Utiliser math.h pour une racine carrée

```
#include <stdio.h>
#include <math.h> /* indispensable pour utiliser sqrt() */

int main()
{
    float cote = 9.0;
    float racine;
    racine = sqrt(cote); /* sqrt calcule la racine carree */
    printf("Racine de 9 = %f\n", racine);
    printf("2 puissance 10 = %f\n", pow(2, 10));
    return 0;
}
```

Sans la ligne `#include <math.h>`, le compilateur ne connaît pas `sqrt()` et signale une erreur.

## 2.3 La fonction `main()` : le point d'entrée du programme

**Intuition.** Une pièce de théâtre peut contenir plusieurs actes, mais le rideau se lève toujours au même endroit : le début de la pièce. De même, un programme C peut contenir de nombreuses fonctions, mais l'exécution commence toujours au même endroit : la fonction `main()` (qui signifie « principale » en anglais).

**PROPRIÉTÉ** Rôle de la fonction `main()` — admise

Tout programme C commence son exécution par la première instruction de la fonction `main()`. Cette fonction est obligatoire et unique : si elle est absente ou mal orthographiée (`Main`, `mane...`), la compilation échoue. Son corps est délimité par une accolade ouvrante { et une accolade fermante }. L'instruction `return 0;` placée à la fin indique au système d'exploitation que le programme s'est terminé sans erreur.

**ATTENTION** L'erreur n°1 du débutant : point-virgule et accolades

Les deux premières causes d'erreurs de compilation chez les élèves sont :

- l'oubli du point-virgule en fin d'instruction : en C, chaque instruction se termine par ; ; le compilateur signale alors souvent l'erreur sur la ligne suivante, ce qui déroute les débutants ;
- les accolades mal appariées : chaque { ouvert doit avoir exactement un } fermant. Indente ton code (décale le contenu des accolades vers la droite) : les environnements comme Code ::Blocks le font automatiquement et les oublis deviennent visibles immédiatement.

Autre piège classique : le C est **sensible à la casse** : `Main`, `MAIN` et `main` sont trois identificateurs différents. Seul `main` (tout en minuscules) est reconnu.

## 2.4 Types de base et déclaration des variables

**Intuition.** En algorithmique, avant d'utiliser une variable, tu la declares dans la partie Variable : cela revient à réserver une case mémoire étiquetée d'un nom et dimensionnée pour un type de donnée. Le C impose exactement la même discipline : **toute variable doit être déclarée avant d'être utilisée**, avec un type qui précise la nature des valeurs qu'elle peut contenir.

**DÉFINITION** Types de base du langage C

Les trois types de base au programme sont :

- `int` : nombre **entier** (ex. 25, -7) — correspond au type algorithmique Entier ;
- `float` : nombre **réel** (à virgule flottante, ex. 12,75 noté 12.75 en C) — correspond au type Réel ;
- `char` : un **caractère** unique, écrit entre apostrophes (ex. 'A') — correspond au type Caractère.

| Type algorithmique | Type C | Format<br><small>printf/scanf</small> | Exemple de déclaration |
|--------------------|--------|---------------------------------------|------------------------|
| Entier             | int    | %d                                    | int age;               |
| Réel               | float  | %f                                    | float moyenne;         |
| Caractère          | char   | %c                                    | char mention;          |

Correspondance entre types algorithmiques, types C et formats d'affichage/saisie (les formats seront détaillés dans la section suivante).

### PROPRIÉTÉ Déclaration de variables

La déclaration d'une variable suit la syntaxe :

```
type nom_de_la_variable ;
```

On peut déclarer plusieurs variables du même type sur une ligne, séparées par des virgules, et on peut **initialiser** une variable dès sa déclaration :

```
int age;           /* variable entiere non initialisee */
float n1, n2, moyenne; /* trois variables reelles */
char reponse = 'O'; /* declaration avec initialisation */
int effectif = 45;  /* un entier initialise a 45 */
```

Une variable non initialisée contient une valeur imprévisible (le « résidu » de la mémoire) : c'est une source d'erreurs redoutable. Prends l'habitude d'initialiser.

**Règles de nommage des identificateurs.** Le nom d'une variable (ou **identificateur**) doit obéir à des règles strictes :

- il commence par une **lettre** ou par le caractère de soulignement `_` ;
- il ne contient que des lettres (sans accent!), des chiffres et le souligné : `moyenneTrim1` est valide, `moyenne-trim1` et `note élève` ne le sont pas ;
- il ne doit pas être un **mot réservé** du langage (`int`, `return`, `if`, `while`...);
- majuscules et minuscules sont distinguées : `Note` et `note` sont deux variables différentes.

### DÉFINITION Constante

Une **constante** est une donnée dont la valeur ne peut pas changer pendant l'exécution. En C, on la déclare de deux façons :

- avec le mot-clé `const` placé devant le type : `const float TVA = 19.25;` — la constante possède alors un type et toute tentative de modification provoque une erreur de

compilation ;

- avec la directive `#define` placée en tête de fichier, sans type ni point-virgule : `#define PI 3.14` — le préprocesseur remplace alors chaque occurrence de `PI` par `3.14` avant la compilation.

Par convention, les constantes sont souvent nommées en MAJUSCULES.

#### EXEMPLE Constantes dans un contexte camerounais

```
#include <stdio.h>
#define SEUIL 10.0      /* moyenne de passage : 10/20 */

int main()
{
    const float TVA = 19.25; /* taux de TVA en pourcentage */
    float prixHT = 5000.0; /* prix hors taxe en FCFA */
    float prixTTC;
    prixTTC = prixHT * (1 + TVA / 100);
    printf("Prix TTC : %f FCFA\n", prixTTC);
    return 0;
}
```

## 2.5 Affectation, opérateurs et commentaires

**Intuition.** En algorithmique, tu écris  $moyenne \leftarrow (n1 + n2) / 2$ . La flèche n'existe pas sur un clavier : le C la remplace par le signe `=`, qu'il faut lire « reçoit » et non « égale ».

#### PROPRIÉTÉ Affectation et opérateurs arithmétiques

L'**affectation** s'écrit `variable = expression`; la valeur de l'expression de droite est calculée puis rangée dans la variable de gauche. Les opérateurs arithmétiques sont :

| Opérateur | Signification                         | Exemple (avec <code>a = 14, b = 4</code> )                                                                                                    |
|-----------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| +         | addition                              | <code>a + b</code> vaut 18                                                                                                                    |
| -         | soustraction                          | <code>a - b</code> vaut 10                                                                                                                    |
| *         | multiplication                        | <code>a * b</code> vaut 56                                                                                                                    |
| /         | division                              | <code>a / b</code> vaut 3 si <code>a</code> et <code>b</code> sont <code>int</code> (division entière), 3,5 si ce sont des <code>float</code> |
| %         | reste de la division entière (modulo) | <code>a % b</code> vaut 2 ; réservé aux entiers                                                                                               |

**ATTENTION** Division entière et modulo : les pièges

- Si les deux opérandes de `/` sont de type `int`, le résultat est tronqué : `7 / 2` donne 3 et non 3,5. Pour obtenir un réel, au moins un opérande doit être réel : `7.0 / 2` donne 3,5.
- L'opérateur `%` est très utile : `n % 2 == 0` teste si un entier `n` est pair ; `n % 10` donne son dernier chiffre.
- Ne confonds jamais `=` (affectation) avec `==` (test d'égalité, vu à la section 4).

**DÉFINITION** Commentaires

Un **commentaire** est un texte ignoré par le compilateur, destiné aux humains qui lisent le programme. Le C en accepte deux formes :

- `// commentaire` : tout ce qui suit sur la ligne est un commentaire ;
- `/* commentaire */` : commentaire pouvant s'étendre sur plusieurs lignes.

Documenter son programme (rôle des variables, étapes du calcul) est un savoir-être du bon programmeur : relis un de tes programmes trois mois plus tard, et tu comprendras pourquoi.

## 2.6 Exemple chiffré complet : la moyenne de deux notes

Mettons tout cela en musique avec le programme annoncé : saisir deux notes d'un élève, calculer et afficher leur moyenne.

**EXEMPLE** Programme complet : moyenne de deux notes

```
#include <stdio.h>

int main()
{
    /* 1. Declarations : trois variables reelles */
    float n1, n2, m;

    /* 2. Saisie des deux notes */
    printf("Entrez la premiere note : ");
    scanf("%f", &n1);
    printf("Entrez la deuxieme note : ");
    scanf("%f", &n2);

    /* 3. Calcul de la moyenne */
    m = (n1 + n2) / 2;

    /* 4. Affichage du resultat */
```

```
printf("La moyenne est : %f\n", m);

return 0;
}
```

Si l'utilisateur saisit 12 puis 15, le programme calcule  $m = (12 + 15)/2 = 13,5$  et affiche : La moyenne est : 13.500000. Remarque l'ordre immuable : déclarations d'abord, puis saisies, puis traitement, puis sortie — exactement le schéma Entrées / Traitement / Sorties de l'algorithmique.

**EXERCICE RÉSOLU 2****Traduction d'un algorithme en programme C**

Énoncé. On donne l'algorithme suivant, qui calcule le montant TTC d'un article vendu dans une boutique de Yaoundé :

```
Algorithme PrixTTC
Variable prixHT, montant : Reel
Constante TAUX : Reel ← 19.25
Debut
  Ecrire("Prix hors taxe ? ")
  Lire(prixHT)
  montant ← prixHT * (1 + TAUX / 100)
  Ecrire("Montant TTC : ", montant)
Fin
```

Traduire cet algorithme en langage C, en respectant la structure complète d'un programme.

**SOLUTION**

Solution détaillée. On suit le squelette type : inclusion de `stdio.h`, fonction `main()`, déclarations (le type Réel devient `float`, la constante devient `const float`), puis les instructions où Lire devient `scanf` et Écrire devient `printf`, et l'affectation `←` devient `=`.

```
#include <stdio.h>

int main()
{
    const float TAUX = 19.25; /* taux de TVA en % */
    float prixHT, montant; /* variables reelles */

    printf("Prix hors taxe ? ");
    scanf("%f", &prixHT); /* traduction de Lire(prixHT) */

    montant = prixHT * (1 + TAUX / 100); /* affectation */

    printf("Montant TTC : %f\n", montant); /* traduction de Ecrire */

    return 0;
}
```

Vérification mentale : pour un prix hors taxe de 10000 FCFA, le programme calcule  $10000 \times (1 + 19,25/100) = 11925$  et affiche `Montant TTC : 11925.000000`. Chaque instruction se termine par un point-virgule, chaque accolade ouverte est fermée, et la constante `TAUX` ne peut pas être modifiée par erreur dans le programme.

#### À RETENIR L'essentiel de la section

- Tout programme C suit la structure : directives `#include`, puis `int main() { ... return 0; }`.
- `stdio.h` fournit les entrées/sorties (`printf`, `scanf`); `math.h` fournit les fonctions mathématiques (`sqrt`, `pow`).
- L'exécution commence obligatoirement dans `main()`; sans elle, la compilation échoue.
- Les types de base sont `int` (Entier), `float` (Réel), `char` (Caractère); toute variable est déclarée avant usage.
- Une constante se déclare avec `const` (typée) ou `#define` (en tête de fichier).
- L'affectation s'écrit `=`; les opérateurs sont `+`, `-`, `*`, `/` et `%` (modulo, entiers uniquement).
- Chaque instruction se termine par un point-virgule; chaque accolade ouverte doit être fermée; on documente avec `//` et `/* */`.

#### EXERCICE 2 À toi de jouer

Écris un programme C complet qui demande le rayon d'un cercle (un réel), puis calcule et affiche son périmètre  $P = 2\pi r$  et son aire  $A = \pi r^2$ . Tu utiliseras la constante `#define PI 3.14` et tu pourras utiliser `pow(r, 2)` de `math.h` ou simplement `r * r`.

#### CORRIGÉ Exercice 1

```
#include <stdio.h>
#include <math.h>
#define PI 3.14

int main()
{
    float r, perimetre, aire;

    printf("Rayon du cercle ? ");
```

```
scanf("%f", &r);

perimetre = 2 * PI * r;
aire = PI * r * r;      /* ou PI * pow(r, 2) */

printf("Perimetre : %f\n", perimetre);
printf("Aire : %f\n", aire);

return 0;
}
```

Pour  $r = 5$ , on obtient un périmètre de 31,4 et une aire de 78,5. Points de vigilance : ne pas oublier `#include <math.h>` si l'on utilise `pow`, et ne pas mettre de point-virgule après la ligne `#define`.

Maintenant que le squelette du programme est solide, la section suivante détaille les deux fonctions qui font dialoguer ton programme avec l'utilisateur : `scanf()` pour la saisie et `printf()` pour l'affichage, avec leurs formats `%d`, `%f` et `%c`.

### 3 ENTRÉES ET SORTIES : SCANF() ET PRINTF()

Dans la section précédente, nous avons appris à construire le squelette d'un programme C : inclusion des bibliothèques `stdio.h` et `math.h`, fonction `main()`, déclaration des variables et des constantes. Mais un programme qui déclare des variables sans jamais communiquer avec l'utilisateur est comme un vendeur du marché Mokolo qui garderait ses calculs dans sa tête : personne ne sait ce qu'il fait, et personne ne peut lui donner de commande. Pour qu'un programme soit utile, il doit pouvoir **recevoir des données** (les entrées) et **restituer des résultats** (les sorties). En algorithmique, tu connais déjà les deux instructions qui assurent ce dialogue : **Lire** et **Écrire**. Cette section t'apprend à les traduire fidèlement en langage C grâce à deux fonctions de la bibliothèque `stdio.h` : `scanf()` et `printf()`.

#### 3.1 La fonction printf() : traduction de Écrire

**Intuition.** Quand tu écris en algorithmique `Écrire("Bonjour")` ou `Écrire("La moyenne est : ", m)`, tu demandes à la machine d'afficher un message à l'écran. En C, le rôle est joué par la fonction `printf()`, dont le nom signifie `print formatted` (afficher de manière formatée). Le mot « formatée » est essentiel : `printf()` ne se contente pas d'afficher du texte, elle peut insérer la valeur de variables à des endroits précis du message, à condition de lui indiquer, pour chaque variable, à quel endroit et sous quel type elle doit apparaître.

##### DÉFINITION Spécificateur de format

Un **spécificateur de format** est un code commençant par le caractère `%`, placé dans la chaîne de caractères de `printf()` ou `scanf()`, qui indique le type de la donnée à afficher ou à saisir. Les trois spécificateurs exigés au programme sont :

- `%d` pour un entier (type `int`);
- `%f` pour un nombre réel (type `float`);
- `%c` pour un caractère (type `char`).

Chaque spécificateur `%` est remplacé, à l'exécution, par la valeur de la variable correspondante, dans l'ordre où les variables sont listées après la chaîne.

#### PROPRIÉTÉ Syntaxe générale de `printf()`

```
printf("texte avec %d et %f", variable1, variable2);
```

- Le premier argument est une chaîne de caractères entre guillemets doubles `" "`. Elle contient le texte à afficher et autant de spécificateurs `%` qu'il y a de variables à afficher.
- Les arguments suivants sont les variables (ou expressions), séparées par des virgules, dans le même ordre que les spécificateurs.
- La séquence `\n` (newline) provoque un saut de ligne : tout ce qui suit s'affiche sur la ligne suivante. Sans `\n`, les affichages se collent les uns aux autres.
- La ligne se termine obligatoirement par un point-virgule.

#### EXEMPLE Premiers affichages

```
#include <stdio.h>

int main()
{
    int age = 17;           /* un entier */
    float moyenne = 13.75; /* un reel */
    char mention = 'B';    /* un caractere */

    printf("Bonjour la classe de Terminale !\n");
    printf("Age : %d ans\n", age);
    printf("Moyenne : %f\n", moyenne);
    printf("Mention : %c\n", mention);
    /* Plusieurs variables dans un meme printf : */
    printf("Eleve de %d ans, moyenne %f, mention %c\n",
           age, moyenne, mention);
    return 0;
}
```

À l'exécution, l'écran affiche :

```
Bonjour la classe de Terminale !
Age : 17 ans
```

```
Moyenne : 13.750000
Mention : B
Eleve de 17 ans, moyenne 13.750000, mention B
```

Observe que `%f` affiche par défaut six chiffres après la virgule. Pour n'en afficher que deux, on écrit `%.2f` : `printf("Moyenne : %.2f\n", moyenne);` affiche 13.75.

#### ATTENTION Erreur fréquente : format incompatible avec le type

Utiliser `%d` avec une variable de type `float` (ou `%f` avec un `int`) est une erreur classique. Le compilateur ne la signale pas toujours, mais l'exécution affiche des résultats aberrants (zéro, nombres gigantesques, valeurs négatives incompréhensibles). Règle d'or : `int` → `%d`, `float` → `%f`, `char` → `%c`. Vérifie toujours la correspondance une par une.

### 3.2 La fonction `scanf()` : traduction de Lire

**Intuition.** L'instruction algorithmique `Lire(n)` demande à l'utilisateur de taper une valeur au clavier, puis range cette valeur dans la variable `n`. En C, la fonction `scanf()` fait exactement cela, mais avec une subtilité fondamentale : pour ranger une valeur dans une variable, `scanf()` doit connaître l'adresse de cette variable en mémoire, c'est-à-dire le numéro de la « case » où elle est stockée. C'est le rôle du symbole `&` (esperluette), qui se lit « adresse de ».

#### PROPRIÉTÉ Syntaxe générale de `scanf()`

```
scanf("%d", &variable);
```

- Le premier argument est une chaîne contenant le spécificateur de format correspondant au type de la variable (`%d`, `%f` ou `%c`).
- Le second argument est la variable précédée obligatoirement de `&` : `&age` signifie « l'adresse mémoire de la variable `age` ».
- Le programme se met en pause, attend que l'utilisateur tape une valeur suivie de la touche Entrée, puis range la valeur à l'adresse indiquée.
- Cas particulier du caractère : pour lire un `char`, on écrit `scanf(" %c", &c);` (avec un espace avant `%c`). Cet espace « mange » le retour à la ligne (`\n`) resté dans le tampon après la saisie précédente, sinon `scanf()` lirait ce `\n` au lieu du caractère attendu.

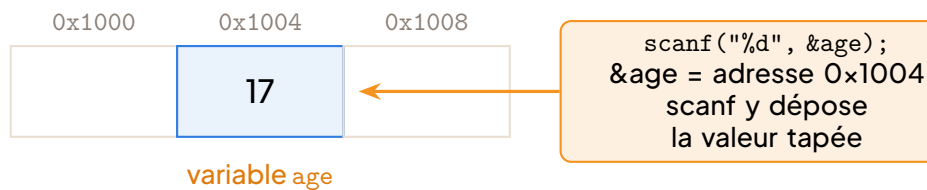


Schéma mémoire : la variable `age` occupe la case d'adresse `0x1004`. L'opérateur `&` fournit cette adresse à `scanf()`, qui peut ainsi y écrire directement la valeur saisie au clavier.

#### MÉTHODE Traduire correctement Lire(x) en C

Pour traduire une instruction `Lire(x)` sans erreur, applique systématiquement les quatre étapes suivantes :

1. Identifier le type de la variable `x` tel qu'il a été déclaré (`int`, `float` ou `char`).
2. Choisir le spécificateur correspondant : `%d` pour `int`, `%f` pour `float`, `%c` pour `char`.
3. Écrire l'appel : `scanf("%d", &x);` ; en n'oubliant ni le `&` devant la variable, ni le point-virgule final. Pour un `char`, écrire `scanf(" %c", &x);` (espace avant `%c`).
4. Guider l'utilisateur : faire précéder chaque `scanf()` d'un `printf()` qui affiche une consigne claire, par exemple `printf("Entrez votre age : ");`, sinon l'écran reste noir et l'utilisateur ne sait pas quoi faire.

#### ATTENTION Erreur fréquente : oublier le & dans scanf()

Écrire `scanf("%d", age);` au lieu de `scanf("%d", &age);` est l'erreur la plus fréquente des débutants. Le programme compile parfois sans message d'erreur, mais à l'exécution il plante (fermeture brutale) ou lit n'importe quoi, car `scanf()` reçoit la valeur de `age` (par exemple 17) et l'utilise comme si c'était une adresse mémoire : il tente d'écrire dans la case numéro 17 de la mémoire, ce qui est interdit. Réflexe à acquérir : dans `scanf`, une variable simple s'écrit toujours avec `&`.

**Saisie de plusieurs valeurs.** Deux styles sont acceptés. Le plus sûr, recommandé au lycée, consiste à faire un **scanf par variable**, chacun précédé de sa consigne :

```
printf("Prix unitaire (FCFA) : ");
scanf("%f", &prix);
printf("Quantité : ");
scanf("%d", &quantite);
```

On peut aussi regrouper dans un **scanf multiple**, en respectant l'ordre des formats et des adresses :

```
printf("Entrez le prix puis la quantité : ");
scanf("%f %d", &prix, &quantite);
```

L'utilisateur tape alors les deux valeurs séparées par un espace, par exemple 2500 3, puis valide par Entrée.

### 3.3 Correspondance systématique algorithme et langage C

Le tableau suivant résume la traduction des instructions d'entrée/sortie. Apprends-le par cœur : il est la clé de toutes les questions d'examen demandant de « traduire un algorithme en C ».

| Algorithme              | Langage C                                 |
|-------------------------|-------------------------------------------|
| Ecrire("Bonjour")       | <code>printf("Bonjour\n");</code>         |
| Ecrire("Moyenne : ", m) | <code>printf("Moyenne : %f\n", m);</code> |
| Lire(n) (n:entier)      | <code>scanf("%d", &amp;n);</code>         |
| Lire(x) (x:réel)        | <code>scanf("%f", &amp;x);</code>         |
| Lire(c) (c:caractère)   | <code>scanf(" %c", &amp;c);</code>        |

Table de traduction des instructions d'entrée/sortie de l'algorithmique vers le langage C.

#### À RETENIR Les quatre réflexes de la traduction

- Écrire devient `printf()`, avec le texte entre guillemets et `\n` pour passer à la ligne.
- Toute variable affichée est annoncée par un spécificateur `%d`, `%f` ou `%c` dans la chaîne, puis citée après la virgule.
- Lire devient `scanf()`, avec le format adapté au type et le `&` devant la variable (espace avant `%c` pour un `char`).
- Chaque instruction se termine par un point-virgule.

### 3.4 Calculs avec `math.h` : `sqrt()` et `pow()`

Les entrées/sorties prennent tout leur sens quand le programme effectue des calculs entre la saisie et l'affichage. La bibliothèque `math.h`, qu'on inclut par `#include <math.h>`, fournit notamment :

- `sqrt(x)` : racine carrée de  $x$  (avec  $x \geq 0$ );
- `pow(x, 2)` :  $x$  élevé à la puissance 2, c'est-à-dire  $x^2$ .

Par exemple, la distance entre deux points  $A(x_A, y_A)$  et  $B(x_B, y_B)$  du plan,  $d = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$ ,

se traduit par :

```
d = sqrt(pow(xB - xA, 2) + pow(yB - yA, 2));
```

De même, pour l'équation du second degré  $ax^2 + bx + c = 0$ , le discriminant  $\Delta = b^2 - 4ac$  s'écrit :

```
delta = pow(b, 2) - 4 * a * c;
```

et, si  $\Delta \geq 0$ , une racine s'obtient par  $x1 = (-b - \text{sqrt}(\text{delta})) / (2 * a)$ ; . Remarque que la multiplication est toujours explicite en C : on écrit  $4 * a * c$  et jamais  $4ac$ .

### EXERCICE RÉSOLU 3 Caisse du marché Mokolo

Énoncé. Une vendeuse du marché Mokolo veut un petit programme qui saisit le prix unitaire d'un article (en FCFA, nombre réel) et la quantité vendue (entier), calcule le montant total, puis l'affiche avec deux décimales. Écris le programme C complet.

#### SOLUTION

Solution détaillée. On déclare `prix` en `float` (format `%f`) et `quantite` en `int` (format `%d`). Le montant est le produit des deux. Chaque `scanf()` est précédé d'un `printf()` de consigne, et le `&` accompagne chaque variable lue.

```
#include <stdio.h>

int main()
{
    float prix;      /* prix unitaire en FCFA */
    int quantite;    /* nombre d'articles */
    float montant;   /* montant total a payer */

    /* Saisies guidees */
    printf("Prix unitaire de l'article (FCFA) :");
    scanf("%f", &prix);      /* %f car prix est float */
    printf("Quantite vendue :");
    scanf("%d", &quantite);  /* %d car quantite est int */

    /* Calcul */
    montant = prix * quantite;

    /* Affichage du resultat, 2 decimales */
    printf("Montant total : %.2f FCFA\n", montant);
    return 0;
}
```

Trace d'exécution : si l'utilisateur tape 2500 puis 3, le programme affiche `Montant total : 7500.00 FCFA`. Points de vigilance : `%f` pour `prix`, `%d` pour `quantite`, `&` devant chaque variable, et `%.2f` pour un affichage propre du montant.

**EXPÉRIENCE** TP guidé : ton premier programme de dialogue

Objectif. Écrire, compiler et exécuter un programme qui demande le nom de la classe, le nombre d'élèves et la moyenne de la classe, puis affiche un récapitulatif.

Matériel et logiciels. Un poste de travail avec l'IDE Code ::Blocks (ou Dev-C++) installé. Manipulation.

1. Lance Code ::Blocks, crée un nouveau projet Console Application en langage C, nommé `dialogue`.
2. Dans le fichier `main.c`, saisis le programme suivant :

```
#include <stdio.h>

int main()
{
    int effectif;
    float moyenne;

    printf("Effectif de la classe : ");
    scanf("%d", &effectif);
    printf("Moyenne de la classe : ");
    scanf("%f", &moyenne);

    printf("La classe compte %d eleves.\n", effectif);
    printf("Sa moyenne generale est %.2f.\n", moyenne);
    return 0;
}
```

3. Compile avec la touche F9 (ou le menu Build), corrige les éventuelles erreurs signalées, puis exécute avec F10 (ou Run).
4. Teste avec les valeurs 52 et 12,5 (selon la configuration du clavier, le séparateur décimal peut être le point : tape alors 12.5).
5. Supprime volontairement le `&` devant `effectif`, recompile et exécute : observe le comportement du programme, puis rétablis le code correct.

Observations attendues. Le programme attend chaque saisie, affiche le récapitulatif formaté, et la suppression du `&` provoque un plantage ou une lecture incohérente à l'exécution.

Interprétation. Ce TP confirme que `printf()` et `scanf()` réalisent le dialogue homme-machine, que le spécificateur doit correspondre au type déclaré, et que le `&` de `scanf()` est indispensable parce qu'il transmet l'adresse mémoire de la variable.

**LESAVAIS-TU ?** Pourquoi le `&` ? Un héritage de 1972

Le langage C a été créé en 1972 par Dennis Ritchie dans les laboratoires Bell, aux États-Unis, pour écrire le système d'exploitation UNIX. Conçu pour être proche de la machine, il donne

au programmeur un contrôle direct sur la mémoire : c'est pourquoi `scanf()` exige l'adresse des variables plutôt que leur simple nom. Cette puissance explique que, plus de cinquante ans après, le C reste le langage de référence pour les systèmes embarqués : il est présent dans les compteurs prépayés, les terminaux de paiement mobile et bien des équipements télécoms utilisés au Cameroun.

### EXERCICE 3 À toi de jouer

1. Traduis en C les instructions algorithmiques suivantes : Écrire("Entrez la note :"), Lire(note) où `note` est un réel, Écrire("Note : ", note).
2. Écris un programme C complet qui saisit les coefficients  $a, b, c$  d'une équation du second degré (trois réels), calcule le discriminant  $\Delta = b^2 - 4ac$  avec `pow()`, et affiche sa valeur.
3. Un élève a écrit `scanf("%f", note);` puis `printf("Note : %d", note);`. Relève les deux erreurs et corrige-les en justifiant.

### CORRIGÉ Exercice n°1

1. Traductions :

```
printf("Entrez la note :\n");
scanf("%f", &note);
printf("Note : %f\n", note);
```

2. Programme du discriminant :

```
#include <stdio.h>
#include <math.h>

int main()
{
    float a, b, c, delta;
    printf("Entrez a, b et c : ");
    scanf("%f %f %f", &a, &b, &c);
    delta = pow(b, 2) - 4 * a * c;
    printf("Delta = %.2f\n", delta);
    return 0;
}
```

3. Première erreur : il manque le `&` devant `note` dans `scanf()` ; sans adresse, la valeur saisie ne peut pas être rangée correctement et le programme risque de planter. Seconde erreur : `note` est un `float`, il faut donc le format `%f` et non `%d` dans `printf()`, sinon l'affichage est aberrant. Code corrigé :

```
scanf("%f", &note);  
printf("Note : %f\n", note);
```

Tu maîtrises désormais le dialogue entre l'utilisateur et le programme : `printf()` pour parler, `scanf()` pour écouter, les formats `%d`, `%f`, `%c` pour se comprendre, et `math.h` pour calculer. La prochaine section te donnera les moyens de faire des choix et des répétitions en C, grâce aux structures de contrôle `if`, `for`, `while` et `do...while`.

## 4 STRUCTURES DE CONTRÔLE EN C

Dans la section précédente, tu as appris à faire dialoguer ton programme avec l'utilisateur grâce à `scanf()` et `printf()`. Mais un programme qui ne fait que lire et afficher des valeurs, ligne après ligne, dans un ordre figé, reste très limité. Or, dans la vraie vie, un programme doit prendre des décisions (« si la moyenne est supérieure ou égale à 10, afficher "Admis" ») et répéter des traitements (« calculer la moyenne de chacun des 45 élèves de la classe »). Ces mécanismes, tu les connais déjà en algorithmique sous les noms **Si...Alors...Sinon**, **Pour**, **TantQue** et **Répéter...Jusqu'à**. Cette section te montre comment les traduire fidèlement en langage C : c'est le cœur de la programmation.

### 4.1 La structure conditionnelle : `if` et `if...else`

**Intuition.** Le proviseur d'un lycée de Yaoundé veut un programme qui affiche « Admis » si la moyenne d'un candidat est au moins égale à 10, et « Ajourné » sinon. Il faut donc que le programme teste une condition et choisisse l'une de deux actions. C'est exactement le rôle du **Si...Alors...Sinon** algorithmique, traduit en C par `if...else`.

#### DÉFINITION Structure conditionnelle en C

La structure conditionnelle permet d'exécuter un bloc d'instructions seulement si une **condition** est vraie. Sa syntaxe générale est :

```
if (condition) {  
    /* instructions exécutées si la condition est vraie */  
} else {  
    /* instructions exécutées si la condition est fausse */  
}
```

La partie `else` est facultative : on peut écrire un `if` seul lorsque rien de particulier n'est à faire quand la condition est fausse. Si le bloc ne contient qu'une seule instruction, les accolades peuvent être omises (mais nous te conseillons de toujours les mettre pour la lisibilité et la prévention d'erreurs).

**EXEMPLE** Décision d'admission

```
float moyenne;
printf("Entrez la moyenne : ");
scanf("%f", &moyenne);

if (moyenne >= 10) {
    printf("Admis(e). Félicitations !\n");
} else {
    printf("Ajourne(e). Courage !\n");
}
```

Si l'utilisateur saisit 12,5, la condition `moyenne >= 10` est vraie et le programme affiche « Admis(e) ». S'il saisit 8, c'est le bloc du `else` qui s'exécute.

## 4.2 Opérateurs de comparaison et opérateurs logiques

Une condition est une expression qui vaut vrai ou faux. En C, elle se construit avec deux familles d'opérateurs.

**DÉFINITION** Opérateurs de comparaison

Les opérateurs de comparaison comparent deux valeurs et produisent un résultat vrai (1) ou faux (0) :

| Opérateur C        | Algorithmique     | Signification         |
|--------------------|-------------------|-----------------------|
| <code>==</code>    | <code>=</code>    | égal à                |
| <code>!=</code>    | <code>≠</code>    | différent de          |
| <code>&lt;</code>  | <code>&lt;</code> | strictement inférieur |
| <code>&gt;</code>  | <code>&gt;</code> | strictement supérieur |
| <code>&lt;=</code> | <code>≤</code>    | inférieur ou égal     |
| <code>&gt;=</code> | <code>≥</code>    | supérieur ou égal     |

**DÉFINITION** Opérateurs logiques

Les opérateurs logiques combinent plusieurs conditions :

- `&&` : **ET** logique — vrai si les deux conditions sont vraies ;
- `||` : **OU** logique — vrai si au moins une condition est vraie ;
- `!` : **NON** logique — inverse la valeur de vérité.

**EXEMPLE** Condition composée

Un élève obtient la mention « Bien » si sa moyenne est comprise entre 14 et 16 (bornes incluses) :

```
if (moyenne >= 14 && moyenne <= 16) {  
    printf("Mention Bien\n");  
}
```

On écrit `&&` et non un seul `&`, et on répète la variable de chaque côté : `moyenne >= 14 && moyenne <= 16`, jamais `14 <= moyenne <= 16` qui ne signifie pas ce qu'on croit en C.

**ATTENTION** Le piège du = dans un test

L'erreur la plus classique du débutant est d'écrire :

```
if (a = b) { ... } /* FAUX : ceci est une AFFECTATION */
```

au lieu de :

```
if (a == b) { ... } /* CORRECT : ceci est une COMPARAISON */
```

En C, `=` est l'opérateur d'affectation : `if (a = b)` recopie la valeur de `b` dans `a`, puis teste cette valeur (`0` = faux, toute valeur non nulle = vrai). Le compilateur ne signale pas toujours d'erreur, mais le programme se comporte de façon aberrante. Retiens : dans une condition, on compare avec `==`.

### 4.3 La boucle `for` : traduction du **Pour**

**Intuition.** Tu veux afficher la table de multiplication de 7, c'est-à-dire répéter 10 fois un calcul en faisant varier un compteur de 1 à 10. Le nombre de répétitions est connu à l'avance : c'est le domaine de la boucle **Pour**, traduite en C par `for`.

**DÉFINITION** La boucle `for`

La boucle `for` répète un bloc d'instructions un nombre déterminé de fois. Sa syntaxe est :

```
for (initialisation ; condition ; incrementation) {  
    /* instructions repetees */  
}
```

Les trois parties jouent chacune un rôle précis :

1. **initialisation** : exécutée une seule fois, avant la première itération (ex. `i = 1`);
2. **condition** : testée avant chaque itération ; tant qu'elle est vraie, le corps s'exécute (ex. `i <= 10`);

3. **incréméntation** : exécutée à la fin de chaque itération, pour faire progresser le compteur (ex. `i++`, qui signifie  $i = i + 1$ ).

**MÉTHODE** Traduire un « Pour  $i$  de 1 à  $n$  »

Pour traduire l'écriture algorithmique Pour  $i$  allant de 1 à  $n$  Faire ... FinPour :

1. Écris `for (i = 1; i <= n; i++)` : départ à 1, condition  $i \leq n$  (inclusive!), pas de 1.
2. Déclare le compteur avant la boucle : `int i;`
3. Place le corps de la boucle entre accolades.
4. Vérifie les bornes sur un petit exemple : pour  $n = 3$ , la boucle doit s'exécuter exactement 3 fois, avec  $i = 1$ , puis  $i = 2$ , puis  $i = 3$ . Si ton test est  $i < n$ , la dernière valeur est perdue : c'est l'erreur de borne la plus fréquente.

**EXEMPLE** Table de multiplication de 7

```
#include <stdio.h>

int main() {
    int i;
    printf("Table de multiplication de 7\n");
    for (i = 1; i <= 10; i++) {
        printf("7 x %d = %d\n", i, 7 * i);
    }
    return 0;
}
```

Exécution :  $i$  part à 1; tant que  $i \leq 10$ , on affiche le produit puis  $i$  augmente de 1. Le programme affiche les 10 lignes, de « 7 x 1 = 7 » à « 7 x 10 = 70 ».

#### 4.4 La boucle `while` : traduction du TantQue

**Intuition.** Un guichetier de la CAMTEL encaisse des clients tant qu'il reste des clients dans la file. Il ne sait pas combien il y en aura : il vérifie la condition avant chaque service. C'est la boucle **TantQue**, traduite par `while`.

**DÉFINITION** La boucle `while`

La boucle `while` répète un bloc tant qu'une condition est vraie, en testant la condition avant chaque itération :

```
while (condition){  
    /* instructions repetees */  
}
```

Conséquence essentielle : si la condition est fausse dès le départ, le corps ne s'exécute jamais (zéro itération possible).

#### EXEMPLE Saisie contrôlée d'une note

```
float note;  
printf("Note (0 a 20) : ");  
scanf("%f", &note);  
  
while (note < 0 || note > 20){  
    printf("Note invalide. Reessayez : ");  
    scanf("%f", &note);  
}  
printf("Note enregistrée : %.2f\n", note);
```

Tant que la note saisie est hors de l'intervalle [0; 20], on redemande la saisie. Si la première saisie est correcte, la boucle ne s'exécute pas du tout.

## 4.5 La boucle `do...while` : traduction du **Répéter...Jusqu'à**

**Intuition.** Un distributeur automatique affiche son menu, lit le choix du client, et recommence jusqu'à ce que le client choisisse « Quitter ». Le menu doit s'afficher au moins une fois : le test vient après le corps. C'est la boucle **Répéter...Jusqu'à**, traduite par `do...while`.

#### DÉFINITION La boucle `do...while`

La boucle `do...while` exécute d'abord le corps, puis teste la condition ; si elle est vraie, on recommence :

```
do{  
    /* instructions repetees */  
} while (condition);
```

Deux particularités à retenir :

- le corps s'exécute au moins une fois, même si la condition est fausse dès le départ ;
- la condition de `while` est une condition de continuation : attention, en algorithmique « Répéter ... Jusqu'à condition d'arrêt », il faut donc souvent inverser la condition lors de la traduction.

Remarque le point-virgule obligatoire après `while (condition);`.

**EXEMPLE** Somme de notes jusqu'à la saisie de -1

Un enseignant saisit les notes de ses élèves ; il tape -1 pour terminer.

```
float note, somme = 0;
int  compteur = 0;

do{
    printf("Note (-1 pour terminer) : ");
    scanf("%f", &note);
    if (note != -1) {
        somme = somme + note;
        compteur++;
    }
} while (note != -1);

if (compteur > 0){
    printf("Moyenne : %.2f\n", somme / compteur);
}
```

Ici, le `do...while` est naturel : on doit saisir au moins une valeur avant de pouvoir décider d'arrêter. La valeur -1 joue le rôle de **sentinelle**.

**PROPRIÉTÉ** Équivalence des trois boucles (admise)

Toute structure répétitive algorithmique (**Pour**, **TantQue**, **Répéter...Jusqu'à**) se traduit en C par `for`, `while` ou `do...while`. Ces trois boucles sont équivalentes en puissance d'expression : tout programme écrit avec l'une peut être réécrit avec une autre. Le choix entre elles est donc une question de clarté et de commodité, pas de possibilité.

**MÉTHODE** Choisir la bonne boucle

1. Le nombre d'itérations est connu à l'avance (parcourir 30 élèves, compter de 1 à n) ⇒ utilise `for`.
2. Le nombre d'itérations est inconnu et il est possible de ne jamais exécuter le corps ⇒ utilise `while`.
3. Le corps doit s'exécuter au moins une fois (menu, saisie avec validation) ⇒ utilise `do...while`.

**ATTENTION** Les boucles infinies

Une **boucle infinie** est une boucle dont la condition ne devient jamais fausse : le pro-

gramme « tourne » sans fin. Deux causes classiques :

1. Oublier l'incrémentation dans un `while` :

```
int i = 1;
while (i <= 10) {
    printf("%d\n", i);
    /* i++ oubliée : i reste à 1, boucle infinie ! */
}
```

2. Mettre un point-virgule juste après la condition :

```
while (i <= 10); /* ce ; est une instruction vide ! */
{
    printf("%d\n", i);
    i++;
}
```

Le `;` seul constitue le corps de la boucle : le programme boucle sur « rien » indéfiniment, et le bloc entre accolades n'est plus lié au `while`.

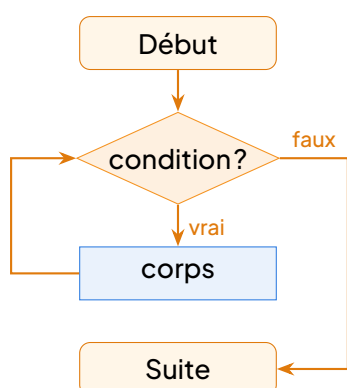
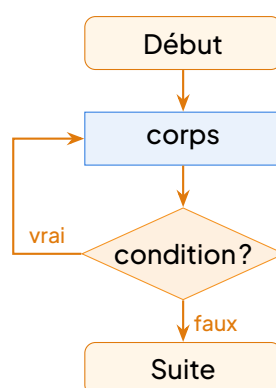
Vérifie toujours que quelque chose, dans le corps, fait évoluer la condition vers « faux ».

## 4.6 Tableau de correspondance et imbrication des structures

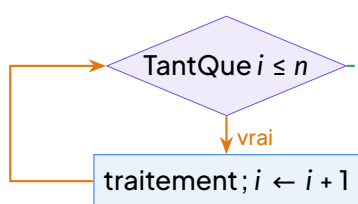
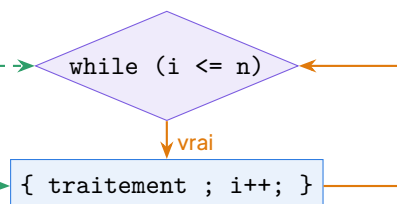
### À RETENIR Correspondance algorithme ↔ langage C

| Écriture algorithmique                 | Traduction en C                                  |
|----------------------------------------|--------------------------------------------------|
| Si condition Alors ... FinSi           | <code>if (condition) { ... }</code>              |
| Si ... Alors ... Sinon ... FinSi       | <code>if (...) { ... } else { ... }</code>       |
| Pour i de 1 à n Faire ... FinPour      | <code>for (i = 1; i &lt;= n; i++) { ... }</code> |
| TantQue condition Faire ... FinTantQue | <code>while (condition) { ... }</code>           |
| Répéter ... Jusqu'à condition          | <code>do { ... } while (!condition);</code>      |
| ET, OU, NON                            | <code>&amp;&amp;,   , !</code>                   |
| <code>a = b</code> (comparaison)       | <code>a == b</code>                              |

Les figures suivantes illustrent visuellement la différence capitale entre `while` et `do...while` : la position du test.

**while : test AVANT****do...while : test APRÈS**

Comparaison des organigrammes : dans `while`, le test précède le corps (0 itération possible); dans `do...while`, le corps s'exécute avant le premier test (au moins 1 itération).

**Algorithme : TantQue****Traduction C : while**

Correspondance structurelle entre le **TantQue** algorithmique et le `while` du langage C : même test, même corps, même retour en boucle.

**Imbrication des structures.** Les structures de contrôle se combinent librement : un `if` peut se trouver dans un `for`, une boucle dans une autre boucle. C'est ce qu'on appelle l'**imbrication**. La règle d'or est l'**indentation** : chaque niveau d'imbrication est décalé vers la droite, ce qui rend le programme lisible et permet de vérifier que chaque accolade ouvrante `{` a bien sa fermante `}`.

**EXERCICE RÉSOLU 4 Compter les admis d'une classe**

Énoncé : écris un programme C qui saisit les moyennes de  $n$  élèves ( $n$  donné par l'utilisateur) et affiche le nombre d'élèves admis (moyenne  $\geq 10$ ).

**SOLUTION**

Solution détaillée : le nombre de passages est connu ( $n$ ), donc on choisit un `for`. À l'intérieur, un `if` teste chaque moyenne : c'est une imbrication `if` dans `for`.

```
#include <stdio.h>

int main() {
    int n, i, admis = 0;
    float moyenne;

    printf("Nombre d'eleves : ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        printf("Moyenne de l'eleve %d : ", i);
        scanf("%f", &moyenne);
        if (moyenne >= 10) {
            admis++; /* un admis de plus */
        }
    }

    printf("Nombre d'admis : %d sur %d\n", admis, n);
    return 0;
}
```

Trace pour  $n = 3$  et les moyennes 12, 7 et 15 : à  $i = 1$ ,  $12 \geq 10$  donc `admis` vaut 1; à  $i = 2$ ,  $7 < 10$ , rien ne change; à  $i = 3$ ,  $15 \geq 10$  donc `admis` vaut 2. Le programme affiche « Nombre d'admis : 2 sur 3 ». Remarque l'initialisation `admis = 0` : sans elle, le compteur partirait d'une valeur imprévisible.

#### EXERCICE 4 À toi de jouer

1. Écris un programme C qui demande un entier  $n$  et affiche la somme  $1+2+\dots+n$  en utilisant un `for`. Vérifie avec  $n = 4$  (résultat attendu : 10).
2. Réécris le programme de saisie contrôlée d'une note (entre 0 et 20) en utilisant un `do...while` au lieu d'un `while`. Quel avantage observes-tu ?
3. Écris un programme qui affiche les tables de multiplication de 1 à 5 (boucles imbriquées : un `for` dans un `for`).

#### CORRIGÉ Exercice : somme des $n$ premiers entiers

```
#include <stdio.h>

int main() {
    int n, i, somme = 0;
    printf("Entrez n : ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
```

```
        somme = somme + i;
    }
    printf("Somme = %d\n", somme);
    return 0;
}
```

Pour  $n = 4$  : `somme` vaut successivement 1, 3, 6, 10 ; le programme affiche « Somme = 10 ». Avec `do...while`, la saisie contrôlée s'écrit `do { scanf(...); } while (note < 0 || note > 20)` ; on évite de dupliquer le `scanf` avant la boucle, car la première saisie fait partie du corps.

#### À RETENIR L'essentiel de la section

- `if (condition) { ... } else { ... }` traduit le **Si...Alors...Sinon** ; le `else` est facultatif.
- Comparaisons : `==, !=, <, >, <=, >=` ; logique : `&&, ||, !`. Jamais `=` dans un test.
- `for (i = 1; i <= n; i++)` traduit le **Pour** : nombre d'itérations connu.
- `while (condition)` traduit le **TantQue** : test avant, 0 itération possible.
- `do { ... } while (condition);` traduit le **Répéter...Jusqu'à** : test après, au moins 1 itération.
- Méfie-toi des boucles infinies (incrémentation oubliée, ; parasite) et soigne toujours l'indentation des structures imbriquées.

Tu maîtrises désormais la logique de n'importe quel programme C. Il ne te reste plus qu'à apprendre à organiser les données en mémoire : direction les tableaux et les enregistrements, que nous étudierons dans la prochaine section avant de les réunir dans l'étude de cas de la recherche séquentielle.

## 5 TABLEAUX ET ENREGISTREMENTS (STRUCT) EN C

Dans la section précédente, tu as appris à traduire en C les structures de contrôle : `if`, `for`, `while` et `do...while`. Tu sais donc désormais répéter des traitements et prendre des décisions dans un programme. Mais il reste un problème de taille, et la coopérative scolaire du lycée de Ngaoundéré le connaît bien : comment manipuler les 40 moyennes d'une classe de Terminale C sans déclarer 40 variables `m1, m2, ..., m40` ? Et comment regrouper sous un même nom les informations d'un élève (son nom, son âge, sa moyenne), qui sont de types différents ? Le langage C répond à ces deux besoins par deux structures de données fondamentales : le **tableau** et l'**enregistrement** (`struct`). C'est l'objet de cette section, qui te préparera à l'étude de cas complète de la recherche séquentielle.

## 5.1 Le tableau : regrouper des éléments de même type

**Intuition : le problème des 40 moyennes.**

Imagine que le proviseur te demande un programme qui calcule la moyenne générale de la classe et affiche la meilleure note. Avec des variables simples, il faudrait déclarer 40 variables distinctes, écrire 40 fois `scanf`, et additionner 40 termes à la main. C'est infaisable en pratique. La solution consiste à utiliser une seule variable qui contient 40 cases numérotées : c'est le tableau.

### DÉFINITION Tableau en C

Un **tableau** est une structure de données qui regroupe, sous un même nom, un nombre fixe d'éléments de même type, accessibles par leur **indice** (position). En langage C, les cases d'un tableau sont contiguës en mémoire et le premier indice est toujours 0 : pour un tableau de  $n$  éléments, les indices valides vont de 0 à  $n - 1$ .

### PROPRIÉTÉ Syntaxe de déclaration d'un tableau

La déclaration d'un tableau se fait dans la zone des déclarations, avant la première instruction :

```
type nom_du_tableau[taille];
```

où `type` est le type commun de tous les éléments (`int`, `float`, `char`...) et `taille` est un entier constant, fixé une fois pour toutes à la déclaration. Exemples :

```
int T[10];          /* tableau de 10 entiers : indices 0 à 9 */
float notes[50];    /* tableau de 50 reels : indices 0 à 49 */
char nom[30];       /* tableau de 30 caracteres (chaîne) */
```

On accède à l'élément d'indice `i` par la notation `T[i]`, qui s'utilise exactement comme une variable ordinaire : on peut y lire, y écrire, l'additionner, la comparer.

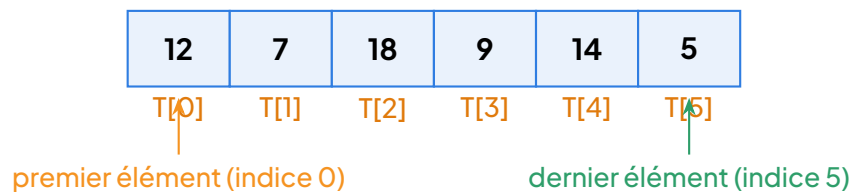
### ATTENTION Le piège de l'indice 0 et du dépassement de taille

Deux erreurs reviennent sans cesse chez les débutants :

- En algorithmique, on numérote souvent les cases de 1 à  $n$ . En C, on numérote de 0 à  $n - 1$ . Le premier élément de `T` est `T[0]`, pas `T[1]`.
- Pour un tableau déclaré `int T[10];`, le dernier indice valide est `T[9]`. Écrire dans `T[10]` ne provoque aucune erreur de compilation : le C ne vérifie pas les bornes ! Le programme écrit alors dans une zone mémoire qui ne lui appartient pas, avec un comportement imprévisible (résultats faux, plantage). C'est une des erreurs les plus graves et les plus fréquentes en C.

Retiens la règle d'or : la boucle de parcours s'écrit toujours `for (i = 0; i < n; i++)` avec un strictement inférieur, jamais `<= n`.

Tableau `int T[6]` en mémoire : 6 cases contiguës



Représentation mémoire d'un tableau `int T[6]` : les cases sont contiguës et numérotées de 0 à 5.

## 5.2 Saisie, affichage et traitements classiques d'un tableau

Un tableau se manipule presque toujours avec une boucle `for`, car l'indice de la boucle coïncide naturellement avec l'indice des cases.

### MÉTHODE Saisir et afficher un tableau de $n$ éléments

1. Déclarer le tableau avec une taille maximale (par exemple `int T[100];`) et une variable `n` pour le nombre réel d'éléments.
2. Saisir `n` en vérifiant qu'il ne dépasse pas la taille maximale.
3. Saisir les éléments un par un : `for (i = 0; i < n; i++) { scanf("format", &T[i]); }` — n'oublie pas le `&` devant `T[i]` : chaque case est une variable comme une autre. Utilise `%d` pour `int`, `%f` pour `float`, `%c` pour `char`.
4. Afficher de la même manière, avec `printf("format", T[i]);` (sans `&` à l'affichage).

### EXEMPLE Saisie et affichage des notes d'une classe

```
#include <stdio.h>

int main() {
    float notes[50]; /* tableau de 50 notes maximum */
    int n, i;

    printf("Nombre d'eleves (max 50) : ");
    scanf("%d", &n);

    /* Saisie des n notes */
```

```
for (i = 0; i < n; i++){
    printf("Note de l'eleve %d : ", i + 1);
    scanf("%f", &notes[i]); /* & obligatoire */
}

/* Affichage des notes saisies */
printf("\nListe des notes : ");
for (i = 0; i < n; i++){
    printf("%.2f ", notes[i]); /* pas de & au printf */
}
printf("\n");
return 0;
}
```

Remarque l'affichage `i + 1` dans le message : pour l'utilisateur, le premier élève est l'élève numéro 1, même si sa note est stockée dans `notes[0]`. C'est une bonne pratique qui masque la numérotation interne à partir de 0.

### Traitements classiques : somme, moyenne, maximum.

Ces trois traitements sont des classiques du Baccalauréat. Ils suivent tous le même schéma : initialiser un accumulateur (ou un candidat), puis parcourir le tableau en mettant à jour.

#### EXEMPLE Somme, moyenne et maximum d'un tableau de notes

```
float somme, moyenne, maxi;
int i;

/* Somme : accumulateur initialise a 0 */
somme = 0;
for (i = 0; i < n; i++){
    somme = somme + notes[i];
}
moyenne = somme / n;

/* Maximum : on part du premier element */
maxi = notes[0];
for (i = 1; i < n; i++){
    if (notes[i] > maxi){
        maxi = notes[i];
    }
}
printf("Moyenne de la classe : %.2f\n", moyenne);
printf("Meilleure note : %.2f\n", maxi);
```

Deux points de vigilance : la somme doit être initialisée à 0 avant la boucle (sinon elle contient une valeur aléatoire), et le maximum est initialisé avec `notes[0]` (le premier élément réel), jamais avec une valeur arbitraire comme 0 — que se passerait-il si toutes les notes étaient négatives ?

### 5.3 Les enregistrements : la structure `struct`

**Intuition : la fiche d'un élève.**

Un tableau impose que tous ses éléments soient du même type. Or la fiche d'un élève de la co-opérative scolaire mélange des types différents : son **nom** (chaîne de caractères), son **âge** (entier), sa **moyenne** (réel). Impossible de tout mettre dans un seul tableau. Il faut une structure qui regroupe des **champs** de types différents sous un même nom : c'est l'enregistrement, traduit en C par le mot-clé `struct`.

#### DÉFINITION Enregistrement (struct) en C

Un **enregistrement** est une structure de données qui regroupe sous un même nom plusieurs **champs** de types éventuellement différents. En C, on le déclare avec le mot-clé `struct`. Chaque champ d'une variable enregistrement est accessible par la notation pointée : `variable.champ`.

#### PROPRIÉTÉ Syntaxe de déclaration et d'utilisation d'une struct

La déclaration se fait en deux temps : on définit d'abord le modèle (le type), puis on déclare des variables de ce type.

```
/* 1. Definition du modele (avant main ou en tete de main) */
struct Etudiant {
    char nom[30]; /* champ nom : chaine de 30 caracteres */
    int age;      /* champ age : entier */
    float moyenne; /* champ moyenne : reel */
};

/* 2. Declaration de variables de ce type */
struct Etudiant e; /* un seul etudiant */
struct Etudiant classe[40]; /* tableau de 40 etudiants */
```

L'accès à un champ se fait avec un point : `e.age`, `e.moyenne`, `e.nom`. Chaque champ s'utilise comme une variable ordinaire de son type.

## struct Etudiant e

nom : char[30] = "Amina"

strcpy(e.nom, "Amina");

age : int = 17

e.age = 17;

moyenne : float = 13,75

e.moyenne = 13.75;

Notation pointée : variable.champ

Un enregistrement struct Etudiant : trois champs de types différents regroupés sous un même nom.

**ATTENTION** Le champ chaîne de caractères ne s'affecte pas avec =

En C, une chaîne de caractères est un tableau de char. Or un tableau ne s'affecte pas avec = : écrire `e.nom = "Amina";` est une erreur. Il faut utiliser la fonction `strcpy` de la bibliothèque `string.h` : `strcpy(e.nom, "Amina");`. À la saisie, on utilise `scanf("%s", e.nom);` sans le `&`, car le nom d'un tableau représente déjà une adresse. En revanche, pour les champs `int` et `float`, le `&` reste obligatoire : `scanf("%d", &e.age);`.

**EXEMPLE** Fiche d'un élève de la coopérative scolaire

```
#include <stdio.h>
#include <string.h>

struct Etudiant{
    char nom[30];
    int age;
    float moyenne;
};

int main(){
    struct Etudiant e; /* la fiche d'un eleve */

    /* Saisie champ par champ */
    printf("Nom de l'eleve : ");
    scanf("%s", e.nom); /* pas de & pour une chaine */
    printf("Age : ");
    scanf("%d", &e.age); /* & obligatoire */
    printf("Moyenne : ");
    scanf("%f", &e.moyenne); /* & obligatoire */

    /* Affichage de la fiche */
    printf("\n--- FICHE DE L'ELEVE ---\n");
    printf("Nom : %s\n", e.nom);
```

```
printf("Age      : %d ans\n", e.age);
printf("Moyenne : %.2f / 20\n", e.moyenne);
if (e.moyenne >= 10){
    printf("Decision : Admis(e)\n");
} else {
    printf("Decision : Ajourne(e)\n");
}
return 0;
}
```

## 5.4 Tableau d'enregistrements : gérer toute une classe

La véritable puissance apparaît quand on combine les deux structures : un **tableau dont chaque case est un enregistrement**. C'est ainsi que l'on modélise une classe entière, le stock d'une boutique ou les abonnés d'un cybercafé.

### PROPRIÉTÉ Déclaration et accès à un tableau d'enregistrements

```
struct Etudiant classe[40]; /* 40 fiches d'eleves */
```

On accède au champ d'un élément en combinant les deux notations : d'abord l'indice entre crochets, puis le point :

```
classe[0].moyenne = 14.5; /* moyenne du 1er eleve */
scanf("%d", &classe[i].age); /* saisie de l'age du i-ieme */
printf("%s", classe[i].nom); /* affichage de son nom */
```

La lecture est naturelle : `classe[i].moyenne` se lit « la moyenne de l'élève d'indice `i` du tableau `classe` ».

### EXERCICE RÉSOLU 5 Moyenne générale et meilleur élève d'une classe

Énoncé. Écrire un programme C complet qui saisit les fiches de  $n$  élèves (nom, âge, moyenne) d'une classe de Terminale, puis affiche la moyenne générale de la classe et le nom du major (l'élève ayant la meilleure moyenne).

#### SOLUTION

Solution détaillée. On utilise un tableau de `struct Etudiant`. Pour le major, on mémorise non pas la meilleure moyenne mais l'indice du meilleur élève, ce qui permet d'afficher ensuite son nom.

```
#include <stdio.h>
```

```
struct Etudiant {
```

```
char nom[30];
int age;
float moyenne;
};

int main() {
    struct Etudiant classe[40];
    int n, i, indiceMajor;
    float somme;

    printf("Nombre d'eleves (max 40) : ");
    scanf("%d", &n);

    /* Saisie des fiches */
    for (i = 0; i < n; i++) {
        printf("\nEleve %d\n", i + 1);
        printf("Nom : ");
        scanf("%s", classe[i].nom);
        printf("Age : ");
        scanf("%d", &classe[i].age);
        printf("Moyenne : ");
        scanf("%f", &classe[i].moyenne);
    }

    /* Moyenne generale : accumulateur */
    somme = 0;
    for (i = 0; i < n; i++) {
        somme = somme + classe[i].moyenne;
    }
    printf("\nMoyenne generale : %.2f / 20\n", somme / n);

    /* Major : on garde l'indice du meilleur */
    indiceMajor = 0;
    for (i = 1; i < n; i++) {
        if (classe[i].moyenne > classe[indiceMajor].moyenne) {
            indiceMajor = i;
        }
    }
    printf("Major de la classe : %s(%.2f / 20)\n",
        classe[indiceMajor].nom,
        classe[indiceMajor].moyenne);
    return 0;
}
```

Trace rapide avec trois élèves : Amina (13,75), Béla (15,50), Carine (11,00). La somme vaut  $13,75 + 15,50 + 11,00 = 40,25$ , d'où une moyenne générale de  $40,25/3 \approx 13,42$ . Le parcours du maximum compare d'abord Béla à Amina ( $15,50 > 13,75$ , donc `indiceMajor` devient 1), puis Carine à Béla ( $11,00 < 15,50$ , rien ne change). Le programme affiche bien « Major de la classe : Béla (15,50 / 20) ».

**EXPÉRIENCE** TP guidé : le fichier des élèves de la coopérative

Objectif : saisir, compiler et exécuter un programme de gestion des fiches élèves sous

Dev-C++ ou Code ::Blocks.

Matériel et logiciels : un poste de travail avec Dev-C++ (ou Code ::Blocks) installé.

Manipulation :

1. Lance l'IDE, crée un nouveau fichier source (Fichier > Nouveau > Fichier source) et enregistre-le sous le nom `cooperative.c`.
2. Recopie le programme de l'exercice résolu ci-dessus, en respectant scrupuleusement l'indentation et les points-virgules.
3. Compile avec la touche F9 (Code ::Blocks) ou F11 (Dev-C++). Corrige les éventuelles erreurs signalées en lisant le numéro de ligne indiqué.
4. Exécute et saisis les fiches de 3 élèves réels de ta classe (accord obtenu!). Note la moyenne générale et le major affichés.
5. Test de robustesse : relance le programme et saisis 45 élèves alors que le tableau est dimensionné à 40. Observe le comportement et tire-en la conclusion.

Observations et interprétation : le programme fonctionne correctement pour  $n \leq 40$ . Pour  $n = 45$ , le programme peut planter ou produire des résultats incohérents : c'est le dépassement de tableau. Un programme robuste doit donc contrôler la saisie de  $n$ , par exemple avec une boucle `do...while` qui impose  $1 \leq n \leq 40$ . Modifie ton programme en ce sens et vérifie qu'il refuse désormais toute valeur hors limites.

#### EXERCICE 5 Stock d'une boutique

La boutique « Chez Maman Françoise » à Yaoundé vend des articles caractérisés par un nom (chaîne de 30 caractères), un prix unitaire en FCFA (entier) et une quantité en stock (entier).

1. Définis en C une structure `struct Article` adaptée.
2. Déclare un tableau `stock` de 100 articles.
3. Écris les instructions qui saisissent les  $n$  articles du stock ( $n \leq 100$ ).
4. Écris les instructions qui calculent et affichent la valeur totale du stock (somme des prix multipliés par les quantités).

#### CORRIGÉ Exercice : stock d'une boutique

```
#include <stdio.h>

struct Article {
    char nom[30];
    int prix;    /* prix unitaire en FCFA */
};
```

```
int quantite; /* quantite en stock */
};

int main() {
    struct Article stock[100];
    int n, i;
    long valeurTotale = 0; /* long : la somme peut etre grande */

    printf("Nombre d'articles (max 100) : ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("\nArticle %d\n", i + 1);
        printf("Nom : ");
        scanf("%s", stock[i].nom);
        printf("Prix unitaire (FCFA) : ");
        scanf("%d", &stock[i].prix);
        printf("Quantite : ");
        scanf("%d", &stock[i].quantite);
    }

    for (i = 0; i < n; i++) {
        valeurTotale = valeurTotale
            + (long) stock[i].prix * stock[i].quantite;
    }
    printf("\nValeur totale du stock : %ld FCFA\n", valeurTotale);
    return 0;
}
```

Note l'usage du type `long` pour la valeur totale : avec des prix de plusieurs milliers de FCFA et des dizaines d'articles, le total dépasserait facilement la capacité d'un simple `int`.

#### À RETENIR L'essentiel de la section

- Un **tableau** regroupe des éléments de même type accessibles par indice ; en C, les indices vont de 0 à `taille - 1` et la taille est fixée à la déclaration.
- Tout parcours de tableau s'écrit `for (i = 0; i < n; i++)`; écrire dans `T[taille]` est un dépassement aux conséquences imprévisibles.
- Les traitements classiques (somme, moyenne, maximum) reposent sur un accumulateur initialisé avant la boucle.
- Une **struct** regroupe des champs de types différents ; l'accès se fait par notation pointée `e.champ`.
- Un tableau d'enregistrements (`struct Etudiant classe[40];`) permet de gérer un ensemble complet de fiches ; on accède à un champ par `classe[i].champ`.
- Pour les champs chaînes de caractères : `scanf("%s", e.nom)` sans `&`, et `strcpy` pour l'affectation.

Tu disposes maintenant de toutes les briques nécessaires : variables, entrées/sorties, structures de contrôle, tableaux et enregistrements. La section suivante les réunit dans une étude de cas complète et emblématique du programme : la recherche séquentielle d'un élément dans un tableau, traduite et exécutée en langage C.

## 6 ÉTUDE DE CAS COMPLÈTE : LA RECHERCHE SÉQUENTIELLE EN C

Dans la section précédente, tu as appris à déclarer et à manipuler en langage C les deux structures de données fondamentales du programme officiel : le **tableau** (`int T[100];`) et l'**enregistrement** (`struct`). Nous savons donc désormais stocker en mémoire une collection de données, par exemple les matricules des membres d'une coopérative. Mais stocker ne suffit pas : il faut pouvoir retrouver une information. C'est exactement le problème que résout l'algorithme de **recherche séquentielle**, étudié en algorithmique dans les leçons précédentes. Cette section est l'aboutissement du module : nous allons traduire cet algorithme, ligne par ligne, en un programme C complet, le compiler, l'exécuter sur plusieurs jeux d'essai et analyser son comportement. C'est le type d'exercice phare du Baccalauréat.

### 6.1 Rappel de l'algorithme de recherche séquentielle

**La situation.** La coopérative des enseignants d'un lycée de Yaoundé conserve dans un tableau les matricules de ses  $n$  membres : par exemple  $T = \{1015, 1087, 1042, 1090\}$ . Le trésorier veut vérifier si le matricule 1042 figure bien dans la liste avant d'encaisser une cotisation. Comment faire ?

**L'intuition.** La méthode la plus naturelle est celle que tu utilises pour retrouver un nom dans une liste affichée au tableau : tu lis le premier nom, puis le deuxième, puis le troisième, et tu t'arrêtes dès que tu as trouvé — ou tu conclus « absent » si tu arrives au bout de la liste sans l'avoir vu. C'est exactement la recherche séquentielle : séquentielle signifie « élément par élément, dans l'ordre ».

#### DÉFINITION Recherche séquentielle

La **recherche séquentielle** (ou recherche linéaire) dans un tableau  $T$  de  $n$  éléments consiste à parcourir le tableau du premier au dernier indice en comparant chaque élément  $T[i]$  à la valeur cherchée  $x$ . Si l'on rencontre un indice  $i$  tel que  $T[i] = x$ , la valeur est trouvée et l'on mémorise sa position ; on peut alors arrêter la recherche. Sinon, après le parcours complet, on conclut que  $x$  est absent du tableau.

**L'algorithme** (écriture algorithmique vue dans les leçons précédentes) :

```
Algorithme Recherche_Sequentielle
Variables : T : tableau[1..100] d'entiers
           n, i, x : entiers
           trouve : booléen
           pos : entier

Début
```

```
Écrire("Nombre d'éléments : ")
Lire(n)
Pour i de 1 à n Faire
    Écrire("T[", i, "] = ")
    Lire(T[i])
FinPour
Écrire("Valeur cherchée : ")
Lire(x)
trouve <- FAUX
i <- 1
Tant que (i <= n) ET (trouve = FAUX) Faire
    Si T[i] = x Alors
        trouve <- VRAI
        pos <- i
    Sinon
        i <- i + 1
    FinSi
FinTantQue
Si trouve = VRAI Alors
    Écrire("Trouvé à la position ", pos)
Sinon
    Écrire("Absent du tableau")
FinSi
Fin
```

**PROPRIÉTÉ Coût de la recherche séquentielle (admis)**

Pour un tableau de  $n$  éléments, la recherche séquentielle effectue au plus  $n$  comparaisons : 1 comparaison si la valeur est en première position (meilleur cas),  $n$  comparaisons si elle est absente ou en dernière position (pire cas). On dit que son coût est linéaire. Son grand avantage : elle fonctionne même si le tableau n'est pas trié, contrairement à d'autres méthodes (comme la recherche dichotomique, hors programme).

## 6.2 Traduction ligne à ligne : de l'algorithme au langage C

Traduire un algorithme en C, c'est appliquer méthodiquement le dictionnaire de correspondances construit dans les sections précédentes. Rappelons les trois points de vigilance propres aux tableaux en C :

- les indices d'un tableau C commencent à 0 : un tableau de  $n$  éléments a les indices 0, 1, ...,  $n-1$ ;
- le booléen n'existe pas en C standard (avant C99) : on le simule par un `int` valant 0 (faux) ou 1 (vrai) ;
- chaque instruction se termine par un point-virgule, et les chaînes de `printf/scanf` utilisent les formats `%d` (entier), `%f` (réel), `&` devant la variable dans `scanf`.

| Étape                               | Algorithme                       | Langage C                          |
|-------------------------------------|----------------------------------|------------------------------------|
| Déclaration du tableau              | T : tableau[1..100] d'entiers    | int T[100];                        |
| Booléen trouvé                      | trouve : booléen                 | int trouve = 0;                    |
| Saisie d'un élément                 | Lire(T[i])                       | scanf("%d", &T[i]);                |
| Boucle de parcours (indices 0..n-1) | Tant que (i < n) ET (non trouve) | for (i = 0; i < n && !trouve; i++) |
| Test d'égalité                      | Si T[i] = x Alors                | if (T[i] == x)                     |
| Verdict                             | Écrire("Trouvé...")              | printf("Trouve...");               |

#### ATTENTION Le piège de l'indice et celui du « = »

Deux erreurs reviennent systématiquement dans les copies et sur les machines :

- Écrire `for (i = 1; i <= n; i++)` comme dans l'algorithme 1-indexé : en C, cela ignore `T[0]` et lit `T[n]` qui n'existe pas (comportement imprévisible, buffer overflow). La boucle correcte est `for (i = 0; i < n; i++)`.
- Écrire `if (T[i] = x)` avec un seul « = » : c'est une affectation, pas une comparaison ! Le C évalue l'affectation comme vraie (si  $x \neq 0$ ) et le programme déclare tout trouvé. La comparaison exige le double égal : `if (T[i] == x)`.

### 6.3 La démarche de traduction d'une recherche

#### MÉTHODE Traduire une recherche séquentielle en C — 6 étapes

1. Déclarer le tableau `T`, les entiers `n`, `i`, `x`, l'indicateur `trouve` (initialisé à 0) et `pos` (initialisé à -1).
2. Saisir la taille effective `n` (avec contrôle  $0 < n \leq \text{TAILLE\_MAX}$ ), puis remplir le tableau avec une boucle `for` et `scanf`.
3. Saisir la valeur cherchée `x`.
4. Parcourir avec `for (i = 0; i < n && !trouve; i++)` : la condition `!trouve` arrête la boucle dès la première occurrence (équivalent du `break` ou du `Tant que` algorithmique).
5. Tester `if (T[i] == x)` : en cas de succès, passer `trouve = 1`; et mémoriser `pos = i`.
6. Afficher le verdict après la boucle : `if (trouve == 1)` afficher « trouvé à la position `pos + 1` » (le `+1` convertit l'indice C en position humaine), sinon afficher « absent ».

**ATTENTION** Erreur fréquente : le verdict prématuré

L'erreur la plus classique consiste à écrire à l'intérieur de la boucle :

```
if (T[i] == x) printf("Trouve");  
else printf("Absent"); /* ERREUR : verdict a chaque tour ! */
```

Avec le tableau {1015, 1087, 1042} et  $x = 1042$ , ce code affiche « Absent, Absent, Trouvé » : il juge après le premier élément différent, alors que la valeur est plus loin. Règle d'or : le verdict ne se donne qu'après le parcours complet, grâce à la variable `trouve` qui mémorise le résultat pendant la boucle.

**ATTENTION** Optimisation : arrêter dès la première occurrence

Dans le programme ci-dessous, la condition `i < n && !trouve` dans le `for` (ou un `break` explicite dans le `if`) stoppe la boucle immédiatement après la découverte. Sans cela, la boucle continue inutilement jusqu'au bout et `pos` mémorise la dernière occurrence en cas de doublons. La recherche séquentielle standard s'arrête à la première occurrence.

## 6.4 Le programme complet, commenté

Voici le programme intégral de recherche d'un membre dans le tableau des matricules de la coopérative, prêt à être compilé sous Dev-C++ ou Code ::Blocks.

```

1  #include <stdio.h>  /* bibliotheque des entrees/sorties */
2
3  int main()
4  {
5      /* ---- Declarations ---- */
6      int T[100];      /* tableau des matricules (100 max) */
7      int n;           /* nombre effectif de membres */
8      int i;           /* indice de parcours */
9      int x;           /* matricule recherche */
10     int trouve = 0;   /* 0 = faux, 1 = vrai */
11     int pos = -1;     /* indice ou x a ete trouve (-1 = non trouve) */
12
13     /* ---- Saisie de la taille avec controle ---- */
14     do {
15         printf("Nombre de membres (1 a 100) : ");
16         scanf("%d", &n);
17     } while (n < 1 || n > 100);
18
19     /* ---- Remplissage du tableau ---- */
20     for (i = 0; i < n; i++) {
21         printf("Matricule du membre %d : ", i + 1);
22         scanf("%d", &T[i]);
23     }
24
25     /* ---- Saisie de la valeur cherchee ---- */
26     printf("Matricule recherche : ");
27     scanf("%d", &x);
28
29     /* ---- Recherche sequentielle (arret des la 1ere occurrence) ---- */
30     for (i = 0; i < n && !trouve; i++) {
31         if (T[i] == x) { /* comparaison : double egal ! */
32             trouve = 1;
33             pos = i;
34         }
35     }
36
37     /* ---- Verdict, APRES la boucle ---- */
38     if (trouve == 1)
39         printf("Le matricule %d est membre (position %d).\n",
40             x, pos + 1);
41     else
42         printf("Le matricule %d est absent de la liste.\n", x);
43
44     return 0;
45 }

```

Programme C complet : recherche séquentielle d'un matricule. Lignes 5-11 : déclarations; lignes 14-17 : contrôle de saisie de  $n$ ; lignes 20-24 : remplissage; lignes 30-35 : cœur de la recherche avec arrêt précoce; lignes 38-42 : verdict après la boucle.

#### EXEMPLE Exemple chiffré : recherche du matricule 1042

Soit le tableau  $T = \{1015, 1087, 1042, 1090\}$  (donc  $n = 4$ ) et la valeur cherchée  $x = 1042$ .

La boucle effectue les comparaisons suivantes :

- $i = 0 : T[0] = 1015 \neq 1042 \rightarrow \text{continue}$  ;
- $i = 1 : T[1] = 1087 \neq 1042 \rightarrow \text{continue}$  ;
- $i = 2 : T[2] = 1042 = 1042 \rightarrow \text{trouve} = 1, \text{pos} = 2$ , la condition `!trouve` devient fausse, la boucle s'arrête.

Après la boucle, `trouve == 1`, donc le programme affiche : « Le matricule 1042 est membre (position 3) ». Remarque bien : la valeur est à l'indice 2 en C, mais à la position 3 pour l'utilisateur humain — d'où le `pos + 1` dans le `printf`.

## 6.5 Trace d'exécution : suivre les variables pas à pas

La **trace d'exécution** est un tableau qui consigne, tour de boucle par tour de boucle, les valeurs des variables. C'est l'outil indispensable pour vérifier un programme « à la main » avant de le compiler — et une compétence exigée au Baccalauréat.

| Tour $i$ | $T[i]$ | $T[i] == x ?$ | trouve / pos |
|----------|--------|---------------|--------------|
| 0        | 1015   | Faux          | 0 / -1       |
| 1        | 1087   | Faux          | 0 / -1       |
| 2        | 1042   | Vrai          | 1 / 2        |

Après la boucle (arrêt précoce) : `trouve = 1`,  
`pos = 2`  $\Rightarrow$  Affichage : « trouvé à la position 3 »

Trace d'exécution pour  $T = \{1015, 1087, 1042, 1090\}$  et  $x = 1042$  : la comparaison devient vraie au tour  $i = 2$ , ce qui bascule `trouve` à 1 et stoppe la boucle.

## 6.6 Compilation et jeux d'essai sous Dev-C++ / Code ::Blocks

### EXPÉRIENCE

#### TP : tester la recherche séquentielle avec 4 jeux d'essai

Objectif : compiler le programme complet et vérifier son comportement dans tous les cas possibles.

Logiciel : Dev-C++ 5.11 ou Code ::Blocks 20.03 (avec le compilateur GCC intégré).

Manipulations :

1. Lance l'IDE, crée un nouveau fichier source (Fichier > Nouveau > Fichier source) et recopie le programme complet de la figure précédente.

2. Enregistre le fichier sous le nom `recherche.c` (l'extension `.c` est importante : elle indique au compilateur qu'il s'agit de C, pas de C++).
3. Compile avec la touche `F9` (Dev-C++) ou `Build` (Code ::Blocks). Corrige toute erreur signalée en lisant le numéro de ligne indiqué.
4. Exécute (`F10` ou `Exécuter`) successivement avec les quatre jeux d'essai ci-dessous, en saisissant à chaque fois le tableau `{1015, 1087, 1042, 1090}` ( $n = 4$ ).

Jeux d'essai et observations attendues :

| Cas testé          | Valeur $x$ | Affichage attendu   |
|--------------------|------------|---------------------|
| Présente au début  | 1015       | membre (position 1) |
| Présente au milieu | 1042       | membre (position 3) |
| Présente à la fin  | 1090       | membre (position 4) |
| Absente            | 2000       | absent de la liste  |

Interprétation : un programme correct doit donner la bonne réponse dans les quatre cas. Tester uniquement une valeur présente au milieu ne prouve rien : les cas limites (première position, dernière position, absence) sont précisément ceux qui révèlent les erreurs d'indices et de verdict.

#### EXERCICE RÉSOLU 6

#### Exercice type Bac : compléter et analyser un programme de recherche

Énoncé. On donne le programme incomplet suivant, destiné à chercher une note dans le tableau des notes d'une classe :

```
#include <stdio.h>
int main()
{
    float notes[50];
    int n, i, trouve = 0;
    float x;
    printf("Effectif : ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%f", &notes[i]);
    printf("Note cherchée : ");
    scanf("%f", &x);
    for (i = 0; i < n; i++) {
        if (notes[i] == x)
            trouve = 1;
    }
    if (trouve == 1)
        printf("Note presente.\n");
    else
        printf("Note absente.\n");
    return 0;
}
```

}

1) Pourquoi utilise-t-on `%f` et non `%d` dans les `scanf`? 2) Combien de comparaisons la boucle de recherche effectue-t-elle si  $n = 30$  et si la note cherchée est absente? 3) Que se passe-t-il si l'utilisateur saisit  $n = 60$ ? Proposer une protection. 4) Relever une imperfection majeure dans la condition `if (notes[i] == x)` pour des notes réelles.

**SOLUTION**

Solution détaillée.

1) Le tableau `notes` est déclaré en `float` (notes avec décimales, ex. 12,5). Le format de lecture doit correspondre au type de la variable : `%f` pour un `float`, `%d` pour un `int`. Utiliser `%d` sur un `float` produirait une lecture erronée (corruption de la représentation binaire).

2) La boucle parcourt tous les éléments sans s'arrêter (pas de `break` ni de condition `!trouve`) : elle effectue donc exactement  $n = 30$  comparaisons, que la note soit présente ou non. C'est le pire cas de la recherche séquentielle (au plus  $n$  comparaisons).

3) Le tableau est déclaré avec 50 cases (indices 0 à 49). Si  $n = 60$ , la boucle de saisie écrit dans `notes[50]` à `notes[59]`, qui n'existent pas : c'est un débordement de tableau (buffer overflow), source de plantage ou de corruption de la mémoire. Protection classique : contrôler la saisie avec une boucle `do...while`.

```
do{
    printf("Effectif (1 a 50) : ");
    scanf("%d",&n);
} while (n < 1 || n > 50);
```

Cette boucle `do...while` impose une saisie valide avant de continuer : c'est la traduction directe de la structure « Répéter ... Jusqu'à » de l'algorithmique.

4) Imperfection majeure : comparer des `float` avec `==` est très risqué en raison des erreurs d'arrondi de la représentation binaire (ex. 12,1 peut être stocké comme 12,099999). Deux valeurs « égales » mathématiquement peuvent ne pas l'être bit à bit.

**ATTENTION Comparaison de réels en C**

Ne jamais écrire `if (a == b)` pour des `float` ou `double`. On teste plutôt la différence absolue : `if (fabs(a - b) < EPSILON)` où `EPSILON` est une petite valeur (ex.  $10^{-6}$ ) et `fabs` (valeur absolue) nécessite `#include <math.h>`.

**LESAVAIS-TU ? Le C, moteur discret de la vie quotidienne**

Les logiciels de gestion des écoles (bulletins, effectifs) et des microfinances au Cameroun — comme ceux utilisés par les réseaux de coopératives d'épargne et de crédit (CAMCCUL) ou les établissements de microfinance de Yaoundé et Douala — sont écrits dans des langages directement issus du C (C++, C#, Java). Chaque fois qu'un guichetier tape un nu-

méro de compte pour retrouver un dossier parmi des milliers, un programme exécute une recherche dans un tableau ou un fichier, selon le principe exact que tu viens de programmer. Le langage C lui-même, créé en 1972 par Dennis Ritchie, reste aujourd'hui le socle des systèmes d'exploitation (Windows, Linux) et des équipements réseau qui font fonctionner la fibre optique nationale.

#### À RETENIR L'essentiel de la section

- La **recherche séquentielle** compare chaque élément du tableau à la valeur cherchée, du premier au dernier; elle fonctionne même si le tableau n'est pas trié et coûte au plus  $n$  comparaisons pour  $n$  éléments.
- En C : initialiser `trouve = 0`, parcourir avec `for (i = 0; i < n && !trouve; i++)` (arrêt précoce), comparer avec `if (T[i] == x)` (double égal!), mémoriser `pos = i`, et n'afficher le verdict qu'après la boucle.
- Les indices C vont de 0 à  $n - 1$ ; la « position humaine » s'obtient par `pos + 1`.
- Un programme se valide par des **jeux d'essai** couvrant tous les cas : valeur au début, au milieu, à la fin, et absente — et par une **trace d'exécution** qui suit les variables tour par tour.
- Attention aux réels : ne jamais tester l'égalité exacte (`==`) sur des `float/double`; utiliser une tolérance (`EPSILON`).

#### EXERCICE 6 À toi de jouer

La bibliothèque de ton lycée conserve dans un tableau les codes de ses 200 ouvrages. Écris en langage C un programme complet qui : (a) saisit les codes des ouvrages ; (b) demande le code d'un livre recherché ; (c) affiche sa position s'il existe, ou le message « Ouvrage non répertorié » sinon ; (d) affiche en plus le nombre de comparaisons effectuées avant de trouver le livre (on arrêtera la boucle dès la première occurrence trouvée grâce à la condition `!trouve` dans le `for` ou l'instruction `break`). Prévois quatre jeux d'essai et rédige la trace d'exécution pour l'un d'eux.

## 7 ACTIVITÉ D'INTÉGRATION

## 8 PROGRAMMATION EN LANGAGE C

### 8.1 Étude de cas complète : Gestion de la coopérative « La Solidarité de Mballa II »

#### Objectifs de l'activité

Cette activité d'intégration vise à mobiliser l'ensemble des savoirs et savoir-faire abordés dans la leçon « Programmation en langage C » pour résoudre un problème concret de gestion. À l'issue de cette activité, vous devez être capable de :

- Concevoir la structure de données adaptée (`struct`) pour représenter une entité réelle (un membre de coopérative).
- Écrire un programme C complet, syntactiquement correct, intégrant les bibliothèques standards, la fonction `main`, les déclarations de variables, de constantes et de tableaux.
- Implémenter les entrées/sorties formatées avec `scanf` et `printf` pour interagir avec l'utilisateur.
- Traduire en C les structures de contrôle algorithmiques : boucle `for` pour le parcours et le calcul de total, structure conditionnelle `if/else` pour la validation, boucle `while` ou `for` pour la **recherche séquentielle**.
- Déboguer, compiler et exécuter le programme sous un EDI (Dev-C++ ou Code ::Blocks) et analyser la trace d'exécution.
- Travailler en équipe, structurer le code par étapes successives et présenter les résultats de manière rigoureuse.

#### Situation

La coopérative d'épargne et de crédit « La Solidarité de Mballa II », située dans l'arrondissement de Yaoundé III, souhaite informatiser la gestion de ses adhérents pour remplacer le registre papier tenu par le secrétaire. Le trésorier, M. Fouda, vous confie le développement d'une première version d'un programme en langage C permettant de gérer la liste des membres et leurs cotisations mensuelles.

Le cahier des charges fonctionnel est le suivant :

1. Un membre est identifié par un **matricule** (entier strictement positif, unique), un **nom** (chaîne de caractères sans espace, max 29 caractères + `\0`) et le **montant de sa cotisation mensuelle** (entier en francs CFA, multiple de 500).
2. Le programme doit permettre la saisie de **n** membres (avec  $1 \leq n \leq 50$ ). La saisie se fait de manière interactive : pour chaque membre, on demande le matricule, le nom et la cotisation.
3. Après la saisie, le programme calcule et affiche le **total des cotisations** collectées pour le mois en cours.

4. Le programme propose ensuite une fonctionnalité de **recherche** : l'utilisateur saisit un matricule ; le programme parcourt le tableau à la recherche de ce matricule (recherche séquentielle). S'il le trouve, il affiche le nom du membre et sa cotisation ; sinon, il affiche le message « Membre introuvable ».
5. Le programme s'arrête après une recherche (version simplifiée pour cette première itération).

**Données de test imposées pour la validation collective (4 membres) :**

| Matricule | Nom     | Cotisation (FCFA) |
|-----------|---------|-------------------|
| 101       | MBARGA  | 5000              |
| 205       | ESSOMBA | 10000             |
| 312       | NGONO   | 7500              |
| 408       | ABANDA  | 15000             |

**Total attendu : 37 500 FCFA. Recherches de test :**

- Recherche 1 (succès) : Matricule 312 → Affichage attendu : Nom : NGONO, Cotisation : 7500 FCFA.
- Recherche 2 (échec) : Matricule 999 → Affichage attendu : Membre introuvable.

### Consignes

Travaillez par groupes de 3 à 4 élèves. Répartissez les rôles : Concepteur/Algorithme, Codeur/-Clavier, Testeur/Valideur, Rapporteur/Présentateur. Suivez rigoureusement les étapes ci-dessous.

#### 1. Analyse et conception (sur papier, 15 min) :

- (a) Écrire la déclaration de la structure `Membre` en C.
- (b) Déclarer le tableau capable de contenir jusqu'à 50 membres et les variables nécessaires (compteur, total, variable de recherche, indicateur de trouvaille).
- (c) Écrire en pseudo-code (français structuré) l'algorithme de la saisie, du calcul du total et de la recherche séquentielle.

#### 2. Implémentation sous EDI (Dev-C++ ou Code ::Blocks, 30 min) :

- (a) Créer un nouveau projet « Console Application » nommé `Cooperative_Mballa2`.
- (b) Traduire fidèlement l'algorithme en C. Respecter la syntaxe exacte : `#include <stdio.h>`, `int main(void)`, accolades, points-virgules, `return 0;`.
- (c) **Attention aux entrées chaînes** : Utiliser `scanf("%29s, tab[i].nom);` pour éviter les débordements (pas d'& pour le nom de tableau).
- (d) Compiler (F11 ou Ctrl+F9) et corriger toutes les erreurs de syntaxe (warnings inclus).

#### 3. Test et validation (15 min) :

- (a) Exécuter le programme (Ctrl+F10 ou F9).
- (b) Saisir exactement les 4 membres du tableau de test ci-dessus.
- (c) Vérifier l'affichage du total (37 500).

- (d) Effectuer la recherche 1 (312) et la recherche 2 (999) en relançant le programme deux fois (ou en ajoutant une boucle `do...while` si vous maîtrisez déjà cette structure bonus).
  - (e) Copier-coller la trace complète de la console dans votre rapport.
4. **Présentation orale (5 min par groupe)** : Le rapporteur présente : la structure `struct` choisie, la logique de la boucle de calcul du total, la condition d'arrêt de la recherche séquentielle, et commente la trace d'exécution obtenue.

## 8.2 Ressources à mobiliser

### PROPRIÉTÉ Syntaxe de déclaration d'une structure et d'un tableau de structures

```
// Définition du type (souvent hors main ou en tout début)
struct Membre {
    int matricule;
    char nom[30]; // 29 chars + \0
    int cotisation; // en FCFA
};

// Déclaration d'un tableau de 50 structures (dans main)
struct Membre tab[50];
int n; // nombre effectif de membres
```

Accès aux champs : `tab[i].matricule`, `tab[i].nom`, `tab[i].cotisation`. **struct tableau de structures**

### PROPRIÉTÉ Boucle `for` pour saisie et calcul de somme

```
int total = 0;
for (int i = 0; i < n; i++){
    printf("Membre %d :\n", i+1);
    printf("Matricule : "); scanf("%d", &tab[i].matricule);
    printf("Nom : "); scanf("%29s", tab[i].nom); // Pas de & pour tableau char
    printf("Cotisation : "); scanf("%d", &tab[i].cotisation);
    total = total + tab[i].cotisation; // Accumulateur
}
printf("Total cotisations : %d FCFA\n", total);
```

**boucle for accumulateur scanf printf**

### PROPRIÉTÉ Algorithme de la recherche séquentielle (traduction C)

```
int recherche;
printf("Matricule recherche : "); scanf("%d", &recherche);
```

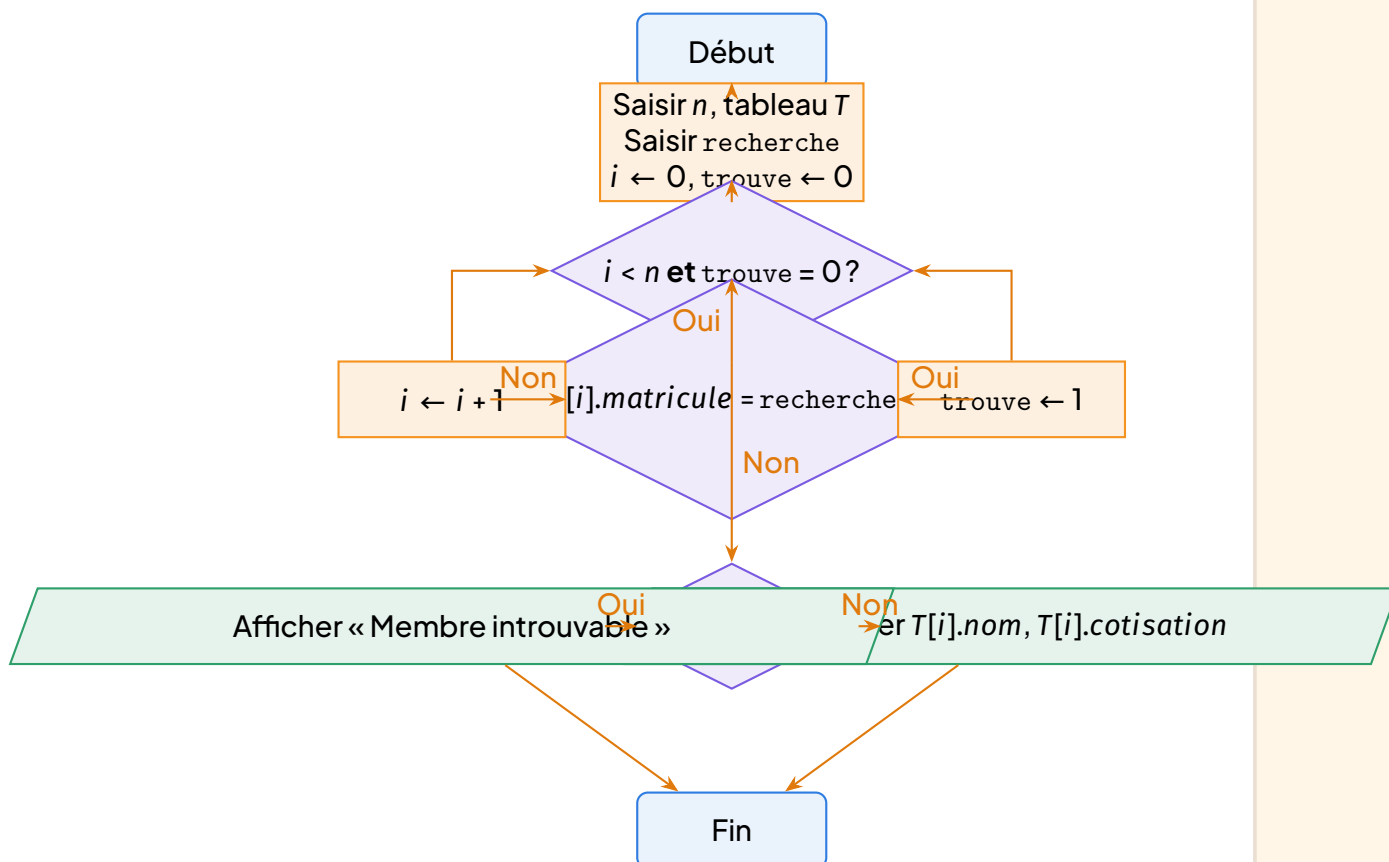
```

int trouve = 0; // 0 = faux, 1 = vrai (booléen "maison")
int i = 0;
while (i < n && trouve == 0) {
    if (tab[i].matricule == recherche) {
        trouve = 1;
    } else {
        i++;
    }
}

if (trouve == 1) {
    printf("Nom : %s, Cotisation : %dFCFA\n", tab[i].nom, tab[i].cotisation);
} else {
    printf("Membre introuvable\n");
}

```

Note : On peut aussi utiliser une boucle for avec break, mais la version while avec indicateur (trouve) est la traduction littérale de l'algorithme standard enseigné. **recherche séquentielle boucle while drapeau (flag)**



Organigramme de la recherche séquentielle avec drapeau trouve

**ATTENTION** Pièges fréquents à éviter

- Oubli de `&` pour les entiers dans `scanf` : `scanf("%d", &tab[i].matricule);` (obligatoire pour modifier la variable). Pas de `&` pour `tab[i].nom` car un nom de tableau est déjà une adresse.
- Dépassement de capacité : Ne pas vérifier que `n <= 50` avant la saisie. Toujours borner la saisie du nom : `%29s` (taille - 1).
- Indice hors limites dans la recherche : La condition `i < n` doit être testée avant d'accéder à `tab[i]`. L'ordre `i < n && trouve == 0` garantit cela (évaluation paresseuse / court-circuit).
- Initialisation de `total` à 0 : Une variable locale non initialisée contient une valeur indéterminée (garbage).
- Encodage console : Sous Windows, les accents peuvent mal s'afficher dans la console de l'EDI. Préférez des messages sans accents (ex : "Membre introuvable") ou utilisez `system("chcp 65001");` en tout début de `main` (spécifique Windows).

### 8.3 Démarche et corrigé commenté

**EXERCICE RÉSOLU 7** Programme complet de gestion de la coopérative « La Solidarité de Mballa II »

Énoncé : Écrire le programme C complet répondant au cahier des charges de la situation (saisie de  $n$  membres dans un tableau de structures, calcul du total des cotisations, recherche séquentielle par matricule).

Solution détaillée :

Étape 1 : Inclusions et définition du type. On inclut `stdio.h` pour `printf/scanf`. On définit `struct Membre` globalement pour qu'il soit connu de `main`. On définit une constante `MAX` pour la taille max du tableau (lisibilité, maintenance).

Étape 2 : Déclarations dans `main`. Tableau `tab[MAX]`, entier `n` (effectif), `total` (accumulateur initialisé à 0), `recherche` (matricule cherché), `i` (indice de boucle), `trouve` (drapeau booléen).

Étape 3 : Saisie de l'effectif  $n$  avec contrôle. Boucle `do...while` pour forcer  $1 \leq n \leq MAX$ . (Structure bonus utile ici).

Étape 4 : Boucle de saisie des  $n$  membres et calcul simultané du total. Boucle `for` de 0 à  $n-1$ . `scanf` pour chaque champ. Accumulation dans `total`. Attention au `&` pour les `int`, pas de `&` pour `nom`, largeur `%29s`.

Étape 5 : Affichage du total.

Étape 6 : Recherche séquentielle. Saisie du matricule cherché. Initialisation `i=0`, `trouve=0`. Boucle `while(i < n && !trouve)`. Test `if (tab[i].matricule == recherche)`. Si trouvé : `trouve=1`, sinon `i++`. Après la boucle, test `if (trouve)` pour affichage différen-

cié.

## Étape 7 : Retour standard.

### SOLUTION

```
#include <stdio.h>

#define MAX 50 // Taille maximale du tableau (constante symbolique)

/* Définition de la structure Membre */
struct Membre {
    int matricule;
    char nom[30]; // 29 caracteres utiles + '\0'
    int cotisation; // Montant en FCFA
};

int main(void) {
    /* 1. Déclarations des variables */
    struct Membre tab[MAX]; // Tableau de structures
    int n; // Nombre effectif de membres
    int total = 0; // Accumulateur : INITIALISATION OBLIGATOIRE A 0
    int recherche; // Matricule à rechercher
    int i; // Indice de parcours
    int trouve; // Drapeau (booléen) : 0 = non trouve, 1 = trouve

    /* 2. Saisie de l'effectif avec controle de borne */
    do {
        printf("Nombre de membres (1 a %d) : ", MAX);
        scanf("%d", &n);
    } while (n < 1 || n > MAX);

    /* 3. Saisie des donnees de chaque membre + Calcul du total */
    printf("\n--- SAISIE DES %d MEMBRES ---\n", n);
    for (i = 0; i < n; i++) {
        printf("\nMembre %d :\n", i + 1);

        printf(" Matricule : ");
        scanf("%d", &tab[i].matricule); // & obligatoire pour int

        printf(" Nom (sans espace, max 29 car.) : ");
        scanf("%29s", tab[i].nom); // PAS de & pour tableau char, limite 29

        printf(" Cotisation (FCFA) : ");
        scanf("%d", &tab[i].cotisation); // & obligatoire pour int

        total = total + tab[i].cotisation; // Accumulation
    }

    /* 4. Affichage du total */
    printf("\n===== \n");
    printf("TOTAL DES COTISATIONS : %d FCFA\n", total);
    printf("===== \n");

    /* 5. Recherche sequentielle par matricule */
    printf("\n--- RECHERCHE D'UN MEMBRE ---\n");
    printf("Entrez le matricule a rechercher : ");
```

```

scanf("%d", &recherche);

/* Initialisation de la recherche */
i = 0;
trouve = 0; // 0 signifie FAUX

/* Boucle de recherche : on arrete des qu'on trouve OU qu'on a tout parcouru */
while (i < n && trouve == 0) {
    if (tab[i].matricule == recherche) {
        trouve = 1; // TROUVE ! On ne incremente pas i, il garde l'indice trouve
    } else {
        i++; // On passe au suivant
    }
}

/* 6. Affichage du resultat de la recherche */
if (trouve == 1) {
    printf("\n>>> MEMBRE TROUVE <<<\n");
    printf("Nom : %s\n", tab[i].nom);
    printf("Cotisation : %d FCFA\n", tab[i].cotisation);
} else {
    printf("\n>>> MEMBRE INTROUVABLE <<<\n");
}

return 0; // Fin normale du programme
}

```

Listing 1 – Code source complet : cooperative-mballa2.c

#### Commentaires techniques sur le corrigé :

- Ligne 4 : `#define MAX 50` évite les « nombres magiques » dans le code. Si la coopérative grandit, on change une seule ligne.
- Lignes 7–11 : La `struct` est définie hors de `main` (portée globale), pratique si on devait créer des fonctions plus tard.
- Ligne 18 : `int total = 0;` Crucial. Sans initialisation, `total` contient une valeur aléatoire (mémoire non initialisée), le calcul serait faux.
- Ligne 27 : `scanf("%29s", tab[i].nom);` La largeur 29 protège contre le débordement de tampon (buffer overflow) si l'utilisateur tape un nom trop long. `tab[i].nom` se dégrade en pointeur (`char *`), donc pas de `&`.
- Ligne 32 : `total = total + tab[i].cotisation;` C'est le schéma classique d'accumulateur (somme).
- Lignes 44–51 : Implémentation canonique de la recherche séquentielle avec drapeau (`trouve`) et indice (`i`). La condition `i < n && trouve == 0` utilise l'évaluation en court-circuit : si `i == n`, la seconde partie n'est pas évaluée, évitant l'accès `tab[n]` hors limites.
- Ligne 54 : `if (trouve == 1).` Comme `i` n'a pas été incrémenté lors de la découverte, il pointe toujours sur l'élément trouvé.

## 8.4 Bilan de l'activité

Cette activité d'intégration a permis de valider l'acquisition des compétences fondamentales de la programmation en C au niveau Terminale :

- **Modélisation de données** : La traduction d'une entité réelle (membre de coopérative) en une `struct` C regroupe des données hétérogènes (entier, chaîne, entier) sous un nom unique. C'est la base de la programmation modulaire.
- **Gestion de collections** : L'utilisation d'un tableau de structures (`struct Membre tab[50]`) permet de manipuler une liste d'enregistrements de taille connue à la compilation (allocation statique), concept précurseur des allocations dynamiques (pointeurs, `malloc`) vues en études supérieures.
- **Traduction algorithmique rigoureuse** : Le passage du pseudo-code (français structuré) au C impose une discipline syntaxique stricte : typage explicite, gestion des adresses pour `scanf`, délimitation des blocs par accolades, terminaison des instructions par point-virgule. L'erreur de syntaxe unique bloque la compilation, forçant la rigueur.
- **Logique de parcours et de recherche** : La boucle `for` pour la saisie/somme (parcours complet borné) et la boucle `while` avec drapeau pour la recherche (parcours conditionnel à arrêt anticipé) illustrent deux patrons de conception (design patterns) fondamentaux : parcours complet vs recherche avec arrêt précoce.
- **Validation par le test** : L'exécution avec le jeu de données de Mballa II (4 membres, total 37 500, recherche 312 et 999) a servi de jeu de tests de non-régression minimal. La confrontation de la trace console aux résultats attendus est la seule preuve de correction fonctionnelle.
- **Contexte professionnel simulé** : Le travail en groupe avec rôles définis (concepteur, codeur, testeur, rapporteur) mime l'organisation d'une équipe de développement logiciel (cycle en V simplifié : spécification → code → test → livraison/démo).

### À RETENIR

### Compétences validées par cette activité

1. Savoir déclarer une `struct` et un tableau de cette structure.
2. Savoir saisir des données dans un tableau de structures via `scanf` (gestion correcte de `&` et largeur pour les chaînes).
3. Savoir parcourir un tableau avec `for` pour effectuer un traitement cumulatif (somme, max, min, affichage).
4. Savoir implémenter la recherche séquentielle en C avec un drapeau (`int trouve = 0;`) et une boucle `while` à condition composée (`i < n && !trouve`).
5. Savoir compiler, corriger les erreurs, exécuter et valider un programme C complet sous Dev-C++ ou Code ::Blocks.

**LESAVAIS-TU ? Le langage C au Cameroun : de l'enseignement à l'industrie**

Le langage C, créé au début des années 1970 par Dennis Ritchie pour réécrire UNIX, reste le « latin » de l'informatique moderne. Au Cameroun, il est au cœur de la formation en Terminale C/D/E car il impose la compréhension de la mémoire (pointeurs, tableaux, structures) sans filet de sécurité (pas de garbage collector).

- **Systèmes embarqués & IoT** : Les compteurs intelligents (prepaid meters) d'ENEO, les bornes de paiement mobile (MTN MoMo, Orange Money), les systèmes de contrôle d'accès biométrique dans les banques (BICEC, Afriland, UBA) tournent sur des micro-contrôleurs programmés en C/C++.
- **Infrastructures télécoms** : Les équipements CAMTEL (commutateurs, routeurs fibre optique, OLTs/ONUs) et les stations de base 4G/5G des opérateurs mobiles utilisent du firmware C temps réel.
- **Logiciels critiques** : Le noyau Linux (serveurs web, cloud, Android) est en C. Maîtriser le C en Terminale, c'est ouvrir la porte aux métiers d'ingénieur système, développeur embarqué, administrateur réseau, ou chercheur en algorithmique.

Ce programme « Coopérative Mballa II », bien que simple, utilise exactement les mêmes briques (`struct` pour la modélisation, tableau pour le stockage, boucle pour le traitement, recherche pour l'accès) que les modules de gestion d'abonnés dans un système de facturation télécom réel.

## 9 MÉTHODES ET EXERCICES RÉSOLUS

## 10 PROGRAMMATION EN LANGAGE C

### LESAVAIS-TU ? Du code source à l'exécutable : compilation et IDE

En Terminale, tu écris du code source en langage C (fichiers `.c`). Ce texte lisible par l'humain doit être traduit en langage machine (binaire) pour être exécuté par le processeur. Cette traduction se fait en trois phases successives, gérées automatiquement par l'IDE (Environnement de Développement Intégré) :

1. **Préprocessing** : le préprocesseur traite les directives commençant par `#` (`#include`, `#define`). Il insère le contenu des fichiers d'en-tête (`.h`) et remplace les constantes symboliques par leurs valeurs.
2. **Compilation proprement dite** : le compilateur traduit le code C prétraité en code assembleur (fichier `.s`), puis en code objet (fichier `.o` ou `.obj`), binaire mais non encore exécutable (adresses mémoire relatives).
3. **Édition de liens (linking)** : l'éditeur de liens combine ton code objet avec les bibliothèques standard (ex. `libc` pour `printf`, `scanf`) et résout les adresses absolues pour produire l'exécutable (`.exe` sous Windows, sans extension sous Linux).

Les IDE officiels au programme sont Dev-C++ (léger, basé sur GCC/MinGW) et Code ::Blocks (multiplateforme, projet-oriented). Ils intègrent éditeur coloré, compilateur, débogueur et gestion de projet. Au Bac, on attend que tu saches créer un projet Console Application, écrire le code, compiler (F9 ou F11) et corriger les erreurs signalées (ligne, type : syntax error, undefined reference, warning).

### 10.1 Méthode 1 : Écrire la structure complète d'un programme C

#### MÉTHODE Écrire la structure complète d'un programme C

Pour rédiger un programme C complet et correct au Bac, suis toujours les étapes suivantes, dans cet ordre :

1. Inclure les bibliothèques nécessaires en tout premier : `#include <stdio.h>` (obligatoire pour `printf` et `scanf`) et `#include <math.h>` si tu utilises des fonctions mathématiques (`sqrt`, `pow`, `cos`, ...).
2. Déclarer les constantes éventuelles avec `#define` (constantes de préprocesseur, sans point-virgule) ou `const type nom = valeur;` (variables en lecture seule, typées).

3. Ouvrir la fonction principale : `int main() {` . Tout programme C commence son exécution dans `main`. Le type de retour `int` indique au système d'exploitation si l'exécution a réussi (`return 0;`) ou échoué (autre valeur).
4. Déclarer toutes les variables en début de bloc (norme C90 exigée au Bac), avec un type adapté : `int` (entier), `float` (réel simple précision), `double` (réel double précision), `char` (caractère ou petit entier).
5. Écrire les instructions : affichages (`printf`), saisies (`scanf`), calculs, structures de contrôle. Chaque instruction se termine par un point-virgule ; .
6. Terminer par `return 0;` puis l'accolade fermante `}`.

Réflexes à retenir : le C est sensible à la casse (`Main` ≠ `main`) ; indente le code (2 ou 4 espaces par niveau) pour la lisibilité ; commente les parties non triviales (`// commentaire` ou `/* ... */`).

**EXERCICE RÉSOLU 8****Premier programme complet : salaire net d'un employé**

Un employé d'une entreprise de Douala perçoit un salaire de base et une prime de transport fixe de 15000 FCFA. On lui retient 4,2 % du salaire de base pour la CNPS. Écrire un programme C complet qui demande le salaire de base, calcule et affiche le salaire net.

**SOLUTION**

Raisonnement. Le salaire net se calcule par :  $\text{net} = \text{base} + \text{prime} - \text{retenue}$ , avec  $\text{retenue} = 0,042 \times \text{base}$ . La prime est une constante ; la base est saisie au clavier (type `float` car un salaire peut avoir des décimales).

Programme complet :

```
#include <stdio.h>

#define PRIME 15000.0f /* prime de transport fixe en FCFA (suffixe f pour float) */

int main()
{
    float base, retenue, net;

    printf("Entrez le salaire de base : ");
    scanf("%f", &base);

    retenue = 0.042f * base;
    net = base + PRIME - retenue;

    printf("Retenue CNPS : %.2f FCFA\n", retenue);
    printf("Salaire net : %.2f FCFA\n", net);

    return 0;
}
```

Vérification. Pour une base de 100000 FCFA : retenue =  $0,042 \times 100000 = 4200$  FCFA ;  
 net =  $100000 + 15000 - 4200 = 110800$  FCFA. Le programme affiche bien 110800.00 FCFA.  
 Conclusion. Le programme respecte la structure canonique : bibliothèque, constante,  
 main, déclarations, entrées, traitement, sorties, return 0.

## 10.2 Méthode 2 : Maîtriser les entrées/sorties scanf() et printf()

### MÉTHODE Utiliser correctement scanf() et printf()

1. Choisir le bon format selon le type de la variable :

| Type             | printf       | scanf                                    |
|------------------|--------------|------------------------------------------|
| int              | %d ou %i     | %d                                       |
| float            | %f (ou %.2f) | %f                                       |
| double           | %f (ou %lf)  | %lf                                      |
| char (caractère) | %c           | %c (attention aux espaces/retours ligne) |
| char[] (chaîne)  | %s           | %s (ou %29s pour limiter)                |

2. Dans scanf, ne jamais oublier l'esperluette & devant le nom de la variable (sauf pour les tableaux/chaînes) : `scanf("%d", &age);`. Sans &, le programme reçoit une valeur au lieu d'une adresse : comportement indéfini (plantage ou résultat aberrant).
3. Dans printf, pas de & : on affiche la valeur : `printf("%d", age);`.
4. Mettre autant de formats que de variables, dans le même ordre : `printf("%d et %f", n, x);` / `scanf("%d %f", &n, &x);`.
5. Formater l'affichage si besoin : %.2f arrondit un réel à 2 décimales ; \n provoque un saut de ligne ; \t une tabulation.
6. Toujours guider l'utilisateur : faire précéder chaque scanf d'un printf explicite ("Entrez ... : ").

Correspondance à retenir : printf traduit l'instruction algorithmique **Écrire**, scanf traduit **Lire**.

### EXERCICE RÉSOLU 9 Traduction Lire/Écrire : moyenne d'un élève

Traduire en C l'algorithme suivant :

Algorithme Moyenne  
 Variables m1, m2, moy : Reel

```
Debut
  Ecrire("Entrez les deux notes : ")
  Lire(m1, m2)
  moy <- (m1 + m2) / 2
  Ecrire("La moyenne est : ", moy)
Fin
```

**SOLUTION**

Raisonnement. Chaque `Ecrire` devient un `printf`, chaque `Lire` devient un `scanf`. Les notes sont des réels : type `float`, format `%f`. Attention : la division  $(m1 + m2) / 2$  porte sur des `float`, donc le résultat est bien réel (pas de division entière).

Traduction :

```
#include <stdio.h>

int main()
{
    float m1, m2, moy;

    printf("Entrez les deux notes : ");
    scanf("%f %f", &m1, &m2); /* deux variables, deux & */

    moy = (m1 + m2) / 2.0f; /* 2.0f force la division réelle */

    printf("La moyenne est : %.2f\n", moy);

    return 0;
}
```

Vérification. Pour  $m1 = 12$  et  $m2 = 15$  :  $moy = \frac{12 + 15}{2} = 13,5$ . Le programme affiche La moyenne est : 13.50.

Conclusion. La traduction est directe ligne à ligne ; les points critiques sont le format `%f` et les deux esperluettes dans `scanf`.

### 10.3 Méthode 3 : Traduire les structures de contrôle (si, pour, tant que)

**MÉTHODE** Traduire les structures de contrôle d'un algorithme en C

1. Identifier la structure dans l'algorithme : alternative (Si ...Alors ...Sinon), boucle bornée (Pour) ou boucle conditionnelle (Tant que, Répéter ...Jusqu'à).
2. Appliquer la correspondance de syntaxe :
  - Si cond Alors ...Sinon ...FinSi  $\rightarrow$  `if (cond) { ...} else { ...}` (le `else` est optionnel).
  - Pour i de 1 à n Faire ...FinPour  $\rightarrow$  `for (i = 1; i <= n; i++) { ...}` (indices 1 à n)

pour l'utilisateur) ou `for (i = 0; i < n; i++)` (indices 0 à  $n-1$  pour tableaux).

- Tant que `cond` Faire ...FinTantQue  $\rightarrow$  `while (cond) { ...}` (test avant l'exécution, peut ne jamais entrer).
- Répéter ...Jusqu'à `cond`  $\rightarrow$  `do { ...} while (!cond);` (test après, au moins une exécution). Attention : l'algorithme s'arrête quand `cond` devient vraie; le `while C` continue tant que sa condition est vraie  $\Rightarrow$  il faut inverser la condition (non logique !).

3. Mettre la condition entre parenthèses : c'est obligatoire en C (`if (note >= 10)`).
4. Utiliser les opérateurs C : égalité `==` (et non `=`), différent `!=`, inférieur/supérieur `<` `>` `<=` `>=`, et logique `&&` (ET), `||` (OU), `!` (NON).
5. Ouvrir et fermer les accolades `{ }` pour tout bloc de plusieurs instructions (recommandé même pour une seule instruction).

#### EXERCICE RÉSOLU 10 Boucles imbriquées : somme des notes supérieures à 10

Un professeur du Lycée de Nkongsamba veut calculer la somme des notes supérieures ou égales à 10 parmi  $n$  notes saisies. Écrire le programme C correspondant.

##### SOLUTION

Raisonnement. Le nombre de notes  $n$  est connu à l'avance : on utilise une boucle `for`. À chaque itération, on saisit une note et on teste avec `if` si elle est  $\geq 10$  ; si oui, on l'ajoute à l'accumulateur `somme`, initialisé à 0 avant la boucle.

Programme :

```
#include <stdio.h>

int main()
{
    int n, i;
    float note, somme = 0.0f; /* accumulateur initialise a 0 */

    printf("Nombre de notes : ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++){
        printf("Note %d : ", i);
        scanf("%f", &note);
        if (note >= 10.0f){
            somme = somme + note;
        }
    }

    printf("Somme des notes >= 10 : %.2f\n", somme);

    return 0;
}
```

}

Trace d'exécution. Pour  $n = 4$  et les notes 8, 12, 14, 9 :

| i | note | note $\geq 10$ ? | somme |
|---|------|------------------|-------|
| 1 | 8    | non              | 0.00  |
| 2 | 12   | oui              | 12.00 |
| 3 | 14   | oui              | 26.00 |
| 4 | 9    | non              | 26.00 |

Conclusion. Le programme affiche 26.00, ce qui correspond bien à  $12 + 14 = 26$ . La boucle `for` traduit le *Pour*, le `if` traduit le *Si*.

**EXERCICE RÉSOLU 11****Boucle conditionnelle : saisie contrôlée avec `do...while`**

Écrire un programme C qui demande une note comprise entre 0 et 20 et la redemande tant qu'elle est invalide, puis affiche « Note valide ».

**SOLUTION**

Raisonnement. La saisie doit avoir lieu au moins une fois avant le test : c'est le cas typique du `do...while` (équivalent du Répéter ...Jusqu'à). L'algorithme s'arrête quand la note est valide ; le `while` C continue tant qu'elle est invalide : `condition = (note < 0 || note > 20)`.  
Programme :

```
#include <stdio.h>

int main()
{
    float note;

    do {
        printf("Entrez une note (0 a 20) : ");
        scanf("%f", &note);
    } while (note < 0.0f || note > 20.0f); /* on recommence si invalide */

    printf("Note valide : %.2f\n", note);

    return 0;
}
```

Vérification. Si l'utilisateur tape 25 puis -3 puis 14 : les deux premières valeurs vérifient la condition de poursuite, la boucle recommence ; 14 la rend fausse, on sort et on affiche Note valide : 14.00.

Conclusion. Le `do...while` garantit au moins une exécution du corps de boucle ; ne pas oublier le point-virgule après le `while(...)`.

## 10.4 Méthode 4 : Déclarer et utiliser un tableau en C

### MÉTHODE Déclarer et manipuler un tableau en C

1. Déclarer le tableau avec son type et sa taille entre crochets : `float notes[30];` (30 cases réelles). On utilise de préférence une constante symbolique : `#define N 30` puis `float notes[N];`.
2. Se rappeler que les indices commencent à 0 : le premier élément est `notes[0]`, le dernier `notes[N-1]`. Accéder à `notes[N]` est une erreur grave (dépassement de tableau, buffer overflow).
3. Remplir le tableau avec une boucle `for` de `i = 0` à `i < N` : `scanf("%f", &notes[i]);` (l'esperluette reste obligatoire car `notes[i]` est un `float`, pas un tableau).
4. Parcourir le tableau avec la même boucle pour afficher, sommer, rechercher : `notes[i]` désigne l'élément courant.
5. Ne jamais comparer ou affecter deux tableaux globalement : `tab1 = tab2;` est interdit, `tab1 == tab2` compare les adresses, pas le contenu. On travaille toujours élément par élément.
6. Passer un tableau à une fonction (hors programme Bac mais utile) : il se dégrade en pointeur, la taille doit être passée en paramètre séparé.

### EXERCICE RÉSOLU 12 Tableau : note maximale d'une classe

Écrire un programme C qui saisit les notes de 10 élèves dans un tableau, puis détermine et affiche la note maximale.

#### SOLUTION

Raisonnement. On déclare un tableau de 10 réels. Après le remplissage, on initialise `max` avec le premier élément `notes[0]` (et non 0, car toutes les notes pourraient être négatives dans un autre contexte), puis on parcourt à partir de l'indice 1 en mettant `max` à jour.

Programme :

```
#include <stdio.h>

#define N 10

int main()
{
    float notes[N], max;
    int i;

    for (i = 0; i < N; i++) {
        printf("Note de l'eleve %d : ", i + 1);
```

```
scanf("%f", &notes[i]);
}

max = notes[0];          /* initialisation au premier element */
for (i = 1; i < N; i++){
    if (notes[i] > max){
        max = notes[i];
    }
}

printf("Note maximale : %.2f\n", max);

return 0;
}
```

Vérification. Pour les notes 5, 12, 8, 17, 9, 11, 14, 6, 10, 13 : le parcours donne successivement  $\text{max} = 5, 12, 12, 17, 17, \dots, 17$ . Affichage : 17.00.

Conclusion. Le tableau se remplit et se parcourt avec une boucle `for` de 0 à  $N-1$  ; l'initialisation du maximum au premier élément est le réflexe attendu au Bac.

## 10.5 Méthode 5 : Déclarer et utiliser un enregistrement (struct)

### MÉTHODE Déclarer et utiliser une structure (enregistrement) en C

1. Définir le type enregistrement avec le mot-clé `struct`, généralement avant `main` (portée globale) :

```
struct Eleve {
    char nom[30];
    int age;
    float moyenne;
}; /* point-virgule OBLIGATOIRE apres l'accolade */
```

2. Déclarer une variable de ce type : `struct Eleve e;` ou un tableau : `struct Eleve classe[50];`.
3. Accéder aux champs avec l'opérateur point : `e.age`, `e.moyenne`, `classe[i].nom`.
4. Saisir les champs :
  - Numérique (`int`, `float`) : `scanf("%d", &e.age);` (& obligatoire).
  - Chaîne (`char[]`) : `scanf("%29s", e.nom);` (sans &, car `e.nom` est déjà l'adresse du tableau ; %29s protège contre le débordement pour un tableau de taille 30).
5. Simplifier avec `typedef` : `typedef struct { ... } Eleve;` puis `Eleve e;` (sans `struct`).

**EXERCICE RÉSOLU 13****Enregistrement : fiche d'un livre de bibliothèque scolaire**

La bibliothèque du Lycée Bilingue de Yaoundé gère ses livres. Écrire un programme C qui définit une structure `Livre` (titre, auteur, année d'édition, nombre d'exemplaires), saisit les informations d'un livre et les affiche sous forme de fiche.

**SOLUTION**

Raisonnement. Un livre regroupe des informations de types différents : c'est un **enregistrement**, traduit en C par `struct`. Les chaînes se saisissent avec `%s` (largeur limitée) sans `esperluette` ; l'entier avec `%d` et `esperluette`.

Programme :

```
#include <stdio.h>

struct Livre {
    char titre[50];
    char auteur[30];
    int annee;
    int exemplaires;
};

int main()
{
    struct Livre l;

    printf("Titre : ");
    scanf("%49s", l.titre);      /* pas de &, largeur max 49 */
    printf("Auteur : ");
    scanf("%29s", l.auteur);
    printf("Annee d'edition : ");
    scanf("%d", &l.annee);      /* & obligatoire pour un int */
    printf("Nombre d'exemplaires : ");
    scanf("%d", &l.exemplaires);

    printf("\n--- FICHE LIVRE ---\n");
    printf("Titre      : %s\n", l.titre);
    printf("Auteur     : %s\n", l.auteur);
    printf("Annee      : %d\n", l.annee);
    printf("Exemplaires : %d\n", l.exemplaires);

    return 0;
}
```

Vérification. Saisie : LeSoleil, Manga, 2015, 12. La fiche affichée reprend exactement ces quatre valeurs, champ par champ grâce à l'opérateur point.

Conclusion. La `struct` regroupe des champs hétérogènes ; l'accès se fait par `variable.champ`, et la règle de l'esperluette dépend du type du champ (adresse pour scalaire, pas d'adresse pour tableau/chaîne).

**10.6 Méthode 6 : Traduire la recherche séquentielle en C**

**MÉTHODE** Traduire l'algorithme de recherche séquentielle en C

1. Disposer des données : un tableau `t` rempli, sa taille effective `n` ( $\leq$  taille max), et la valeur cherchée `x`.
2. Initialiser : un indice `i = 0` et un drapeau `trouve = 0` (en C, 0 = faux, toute valeur non nulle = vrai). Optionnel : `pos = -1`.
3. Parcourir avec un `while` à deux conditions d'arrêt : `while (i < n && trouve == 0)` — on s'arrête en fin de tableau (`i == n`) ou dès qu'on a trouvé (`trouve == 1`).
4. Dans la boucle : si `t[i] == x`, mettre `trouve = 1` et mémoriser la position `pos = i`; sinon avancer : `i++`.
5. Après la boucle : tester `trouve` (ou `pos != -1`) pour afficher « trouvé à la position ... » ou « absent du tableau ».
6. Réflexe Bac : la position affichée pour l'utilisateur est souvent `pos + 1` (numérotation humaine à partir de 1), alors que l'indice C est 0-based.

**EXERCICE RÉSOLU 14** Recherche séquentielle : retrouver un matricule

Le secrétariat d'un collège de Bafoussam stocke les matricules de 8 élèves dans un tableau. Écrire un programme C qui saisit les 8 matricules, demande un matricule à rechercher, puis indique s'il existe et à quelle position.

**SOLUTION**

Raisonnement. On applique le schéma de recherche séquentielle : parcours avec double condition d'arrêt, drapeau `trouve`, mémorisation de la position.

Programme :

```
#include <stdio.h>

#define N 8

int main()
{
    int t[N], x, i = 0, trouve = 0, pos = -1;

    for (i = 0; i < N; i++){
        printf("Matricule %d : ", i + 1);
        scanf("%d", &t[i]);
    }

    printf("Matricule a rechercher : ");
    scanf("%d", &x);

    i = 0;
    while (i < N && trouve == 0) {
```

```

    if (t[i] == x) {
        trouve = 1;
        pos = i;      /* on memorise l'indice 0-based */
    } else {
        i++;          /* on avance seulement si non trouve */
    }
}

if (trouve) {
    printf("Matricule trouve a la position %d\n", pos + 1);
} else {
    printf("Matricule absent du tableau\n");
}

return 0;
}

```

Trace d'exécution. Tableau : [105, 210, 133, 408, 97, 55, 312, 76], recherche de  $x = 408$  :

| i | t[i] | t[i] == 408? | Action                                |
|---|------|--------------|---------------------------------------|
| 0 | 105  | non          | i devient 1                           |
| 1 | 210  | non          | i devient 2                           |
| 2 | 133  | non          | i devient 3                           |
| 3 | 408  | oui          | trouve = 1, pos = 3, sortie de boucle |

Le programme affiche `Matricule trouve a la position 4` (car  $\text{pos} + 1 = 3 + 1 = 4$ ).

Conclusion. La recherche séquentielle compare la valeur cherchée à chaque élément, du premier au dernier, et s'arrête dès la première occurrence : c'est la traduction exacte de l'algorithme vu en classe.

#### EXPÉRIENCE

#### TP guidé : Compilation, débogage et recherche séquentielle sous Dev-C++ / Code ::Blocks

Objectif. Maîtriser l'IDE officiel, compiler un programme C complet intégrant tableau, structure et recherche séquentielle, et utiliser le débogueur pas à pas. Matériel / Logiciels. Poste de travail sous Windows/Linux avec Dev-C++ (version 5.11+ recommandée) ou Code ::Blocks (20.03+). Manipulation.

1. **Créer un projet** : File > New > Project > Console Application > C Project. Nommer RechercheMatricules. Enregistrer dans un dossier sans accents ni espaces (ex. C:\TP\_C\RechercheMatricules).
2. **Écrire le code** : Copier le programme de l'exemple résolu « Recherche séquentielle » dans `main.c`. Ajouter `#include <stdlib.h>` pour `system("pause")` si la console se ferme trop vite sous Windows.
3. **Compiler** : Touche F9 (Dev-C++) ou Ctrl+F9 (Code ::Blocks). Observer la fenêtre

Compile Log. En cas d'erreur (soulignée en rouge), double-cliquer pour aller à la ligne fautive. Corriger (ex. point-virgule manquant, & oublié).

4. Exécuter : Touche F10 (Dev-C++) ou Ctrl+F10 (Code ::Blocks). Tester avec le jeu de données de la trace (105, 210, 133, 408, 97, 55, 312, 76 ; recherche 408). Vérifier l'affichage `Position 4`.
5. Déboguer pas à pas : Placer un point d'arrêt (breakpoint) sur la ligne `while (i < N && trouve == 0)` (clic dans la marge gauche ou F5). Lancer en mode Debug (F8 Dev-C++, F8 Code ::Blocks). Observer les variables `i`, `trouve`, `pos`, `t[i]` dans la fenêtre `Watches / Watch`. Avancer pas à pas (F7 / Next line) pour voir l'incréméntation de `i` et l'arrêt sur `trouve = 1`.
6. Tester les cas limites : Rechercher une valeur absente (ex. 999) → message « absent ». Rechercher le premier élément (105) → position 1. Rechercher le dernier (76) → position 8. Saisir un nombre de notes  $n > N$  (ex. 10) : observer le comportement (écrasement mémoire, plantage ou silence) → leçon : toujours vérifier les bornes.

Observations / Interprétation. Le débogueur permet de visualiser l'état mémoire à chaque instruction, confirmant la logique du `while` à double condition. L'absence de contrôle de taille (`if (n > N) n = N;`) est une faille de sécurité classique (dépassement de tampon).

#### ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Oublier l'esperluette & dans `scanf`. Écrire `scanf("%d", age);` au lieu de `scanf("%d", &age);` est l'erreur la plus fréquente : le programme reçoit une valeur au lieu d'une adresse et se comporte de façon imprévisible (souvent crash ou écriture mémoire aléatoire). Exception à connaître : pas de & pour une chaîne de caractères (`scanf("%29s", nom);` car `nom` est déjà l'adresse du premier caractère).
2. Confondre = (affectation) et == (comparaison). Écrire `if (note = 10)` affecte 10 à `note` et la condition vaut 10 (vrai) : le test est faussé, le bloc `if` s'exécute toujours. Dans toute condition, c'est ==.
3. Se tromper d'indices dans les tableaux. Le premier élément est `t[0]` et le dernier `t[N-1]`. Une boucle écrite `for (i = 1; i <= N; i++)` ignore le premier élément et accède à `t[N]` (hors tableau) : résultats faux garantis, comportement indéfini.
4. Oublier le point-virgule après une instruction, après la déclaration d'une `struct` (;), ou après le `while(...)` d'un `do...while`. Le compilateur refuse le programme (erreur de syntaxe) et le correcteur retire des points.
5. Mal initialiser les accumulateurs et drapeaux. Une somme non initialisée à 0 (contient une valeur aléatoire), un maximum initialisé à 0 au lieu de `t[0]` (faux si toutes notes < 0), ou un drapeau `trouve` non remis à 0 avant la recherche (reste à 1 d'une recherche pré-

cédente) produisent des résultats faux même si la boucle est correcte. Toute variable de calcul doit être initialisée avant la boucle qui l'utilise.

#### EXERCICE 7 Entraînement Bac : Gestion d'un stock de matériel informatique

Un cybercafé de Yaoundé gère un stock de 20 souris. Chaque souris a une référence (chaîne de 15 caractères), un prix unitaire (entier en FCFA) et une quantité en stock (entier).

1. Définir la structure `Souris` adaptée.
2. Écrire un programme C qui :
  - Saisit les informations des 20 souris (remplissage d'un tableau de structures).
  - Calcule et affiche la valeur totale du stock (somme des `prix * quantite`).
  - Demande une référence à rechercher et affiche le prix et la quantité correspondants (recherche séquentielle), ou « Référence introuvable ».

#### CORRIGÉ Exercice : Gestion d'un stock de matériel informatique

Structure et programme complet :

```
#include <stdio.h>
#include <string.h> /* pour strcmp si on veut comparer chaines, mais ici == sur int/float */

#define MAX 20
#define TAILLE_REF 15

struct Souris {
    char reference[TAILLE_REF + 1]; /* +1 pour le \0 final */
    int prix;
    int quantite;
};

int main()
{
    struct Souris stock[MAX];
    int i, n = MAX;
    long long valeur_totale = 0; /* long long pour eviter debordement : 20 * 50000 * 100 = 100 000 000 */
    char ref_cherchee[TAILLE_REF + 1];
    int trouve = 0, pos = -1;

    /* 1. Remplissage */
    for (i = 0; i < n; i++) {
        printf("\n--- Souris %d ---\n", i + 1);
        printf("Reference (max %d car) : ", TAILLE_REF);
        scanf("%15s", stock[i].reference); /* largeur limitee */
    }
}
```

```
    printf("Prix unitaire (FCFA) : ");
    scanf("%d", &stock[i].prix);
    printf("Quantite en stock : ");
    scanf("%d", &stock[i].quantite);
}

/* 2. Valeur totale du stock */
for (i = 0; i < n; i++){
    valeur_totale += (long long)stock[i].prix * stock[i].quantite;
}
printf("\nValeur totale du stock : %lld FCFA\n", valeur_totale);

/* 3. Recherche sequentielle par reference */
printf("\nReference a rechercher : ");
scanf("%15s", ref_cherchee);

i = 0;
while (i < n && !trouve){
    /* strcmp renvoie 0 si chaines identiques */
    if (strcmp(stock[i].reference, ref_cherchee) == 0){
        trouve = 1;
        pos = i;
    } else {
        i++;
    }
}

if (trouve){
    printf("Trouve : Prix = %d FCFA, Quantite = %d\n",
        stock[pos].prix, stock[pos].quantite);
} else {
    printf("Reference introuvable.\n");
}

return 0;
}
```

#### Points clés du corrigé :

- `#include <string.h>` nécessaire pour `strcmp` (comparaison de chaînes, car `==` compare les adresses, pas le contenu).
- `long long` pour la valeur totale (produit de deux `int` peut dépasser `int` sur 16 bits, et la somme sur 20 éléments aussi).
- `scanf("%15s", ...)` protège contre le débordement de `reference[16]`.
- Initialisation `valeur_totale = 0` et `trouve = 0` avant les boucles.
- Recherche séquentielle avec `strcmp` et double condition `i < n && !trouve`.

## 11 EXERCICES

### 11.1 Je vérifie mes connaissances

#### EXERCICE 8 QCM : les bases du langage C

Pour chaque question, choisis la (ou les) bonne(s) réponse(s).

1. L'instruction qui permet d'afficher la valeur d'une variable entière `n` en langage C est :

- a) `printf("n");`
- b) `printf("%d", n);`
- c) `scanf("%d", &n);`
- d) `Ecrire(n);`

2. En C, la déclaration correcte d'une variable réelle nommée `moyenne` est :

- a) `int moyenne;`
- b) `float moyenne;`
- c) `real moyenne;`
- d) `float : moyenne;`

3. La bibliothèque à inclure pour utiliser `printf()` et `scanf()` est :

- a) `math.h`
- b) `stdlib.h`
- c) `stdio.h`
- d) `string.h`

4. Le symbole `&` dans `scanf("%d", &age);` sert à :

- a) afficher la valeur de `age`;
- b) indiquer l'adresse mémoire de la variable `age`;
- c) déclarer la variable `age`;
- d) tester l'égalité de deux variables.

#### EXERCICE 9 Vrai ou faux — justifier

Réponds par vrai ou faux, puis justifie ta réponse en une ou deux phrases.

1. En langage C, l'opérateur `=` permet de tester l'égalité de deux valeurs.
2. Tout programme C doit obligatoirement contenir une fonction nommée `main`.

3. En C, le premier élément d'un tableau déclaré `int T[10];` est `T[1]`.
4. La boucle `do...while` exécute son corps au moins une fois, même si la condition est fausse dès le départ.
5. On peut écrire un programme C dans le bloc-notes de Windows, mais il faut un compilateur pour obtenir un programme exécutable.

#### EXERCICE 10 Définitions et vocabulaire

1. Définis les termes suivants : **compilateur**, **IDE**, **enregistrement** (struct).
2. Cite deux environnements de développement (IDE) utilisés pour programmer en langage C au lycée.
3. Quelle est la différence entre un **langage compilé** et un **langage interprété**? À quelle catégorie appartient le langage C?
4. Donne le rôle de la directive `#include <stdio.h>` placée en tête d'un programme C.

#### EXERCICE 11 Correspondance algorithme → langage C

Recopie et complète le tableau suivant en donnant la traduction en langage C de chaque élément d'écriture algorithmique.

| Écriture algorithmique                    | Traduction en C |
|-------------------------------------------|-----------------|
| Variable n : Entier                       |                 |
| Lire(n)                                   |                 |
| Écrire("Bonjour")                         |                 |
| Si (n > 0) Alors ... FinSi                |                 |
| Pour i allant de 1 à 10 Faire ... FinPour |                 |
| Tant que (n < 100) Faire ... FinTantQue   |                 |

## 11.2 Je m'entraîne

#### EXERCICE 12 Mon premier programme complet

Le cybercafé « Espace Jeunes » de Bafoussam facture la connexion Internet à 300 FCFA l'heure. Écris un programme C complet (avec les `#include`, la fonction `main` et les décl-

rations) qui :

1. demande à l'utilisateur le nombre d'heures de connexion (un nombre entier);
2. calcule le montant à payer;
3. affiche le résultat sous la forme : Montant a payer : 1500 FCFA.

#### EXERCICE 13 Traduction de structures de contrôle

On donne l'algorithme suivant :

```
Algorithme Parite
Variable n : Entier
Debut
    Ecrire("Entrez un entier : ")
    Lire(n)
    Si (n mod 2 = 0) Alors
        Ecrire("Pair")
    Sinon
        Ecrire("Impair")
    FinSi
Fin
```

1. Traduis cet algorithme en un programme C complet et correct.
2. Modifie ton programme pour qu'il demande 5 entiers à l'utilisateur (à l'aide d'une boucle `for`) et affiche pour chacun s'il est pair ou impair.

#### EXERCICE 14 Tableau de notes

Écris un programme C qui :

1. déclare un tableau `notes` de 8 réels;
2. remplit le tableau en demandant les 8 notes d'un élève à l'utilisateur;
3. calcule et affiche la somme puis la moyenne des 8 notes;
4. affiche enfin la plus grande note du tableau.

Indication : utilise une boucle `for` pour la saisie, puis une autre boucle pour parcourir le tableau.

#### EXERCICE 15 Trace d'exécution

On donne le programme C suivant :

```
#include <stdio.h>
int main()
{
    int i, s;
    s = 0;
    i = 1;
    while (i <= 5)
    {
        s = s + i;
        printf("i = %d, s = %d\n", i, s);
        i = i + 2;
    }
    printf("Fin : s = %d\n", s);
    return 0;
}
```

1. Construis la table d'exécution du programme (valeurs successives de *i* et de *s*).
2. Donne l'affichage exact produit à l'écran par ce programme.
3. Que se passerait-il si l'on remplaçait *i = i + 2*; par *i = i*; ? Justifie.

### 11.3 Je me prépare au Bac

#### EXERCICE 16 Type Bac — Moyenne générale d'un élève

Au lycée bilingue de Nkongsamba, la moyenne générale d'un élève de Terminale C est calculée à partir de 5 notes sur 20, chacune affectée d'un coefficient entier. On donne l'algorithme suivant :

```
Algorithme MoyenneGenerale
Variables i, coef, sommeCoef : Entier
Variables note, sommeNotes, moyenne : Reel
Debut
    sommeNotes <- 0
    sommeCoef <- 0
    Pour i allant de 1 à 5 Faire
        Ecrire("Note numero ", i, " : ")
        Lire(note)
        Ecrire("Coefficient : ")
        Lire(coef)
        sommeNotes <- sommeNotes + note * coef
        sommeCoef <- sommeCoef + coef
    FinPour
    moyenne <- sommeNotes / sommeCoef
    Ecrire("Moyenne generale : ", moyenne)
    Si (moyenne >= 10) Alors
        Ecrire("Admis(e)")
    Sinon
        Ecrire("Ajourné(e)")
    FinSi
```

Fin

1. Rappelle le rôle de chacune des directives `#include <stdio.h>` et `#include <math.h>` dans un programme C. Laquelle est indispensable ici ?
2. Donne la déclaration en langage C de toutes les variables de cet algorithme, en choisissant les types appropriés.
3. Traduis en langage C la boucle `Pour` de cet algorithme (lignes 6 à 13).
4. Écris le programme C complet correspondant à cet algorithme (directives d'inclusion, fonction `main`, déclarations, instructions).
5. On exécute le programme avec les données suivantes : notes 12, 8, 14, 10, 9 avec les coefficients respectifs 3, 2, 4, 1, 2. Calcule la moyenne affichée et la décision obtenue.

**EXERCICE 17** Type Bac — Complétion et débogage

La boutique « Chez Maman Rose » à Douala utilise un programme C pour gérer ses produits. On définit un produit par son nom, son prix unitaire et sa quantité en stock.

Partie A : complétion. Recopie et complète le programme suivant en remplaçant les pointillés par le code C approprié.

```
#include <stdio.h>

struct Produit
{
    char nom[30];
    ..... prix;          /* prix unitaire en FCFA (reel) */
    int quantite;
};

int main()
{
    struct Produit p1, p2, p3;
    struct Produit max;
    int i;

    /* saisie des 3 produits (code suppose ecrit) */

    max = p1;
    if (p2.prix ..... max.prix)
        max = p2;
    if (.....)
        max = p3;

    printf("Produit le plus cher : %s a %..... FCFA\n",
           max.nom, max.prix);
    return 0;
}
```

1. Quel type C convient pour le champ `prix`? Justifie.
2. Complète les quatre pointillés du programme.
3. Quelle structure de données C permet de regrouper les champs `nom`, `prix` et `quantite`? Donne sa définition complète.

Partie B : débogage. Le programme ci-dessous, censé calculer la moyenne de 3 notes, contient exactement 5 erreurs. Repère-les, numérote-les et donne pour chacune la correction.

```
#include <stdio.h>
int main()
{
    float notes[3];
    float somme, moyenne;
    int i;
    somme = 0;
    for (i = 1; i <= 3; i++)
    {
        printf("Note %d : ", i);
        scanf("%f", notes[i]);
        somme = somme + notes[i]
    }
    moyenne = somme / 3;
    if (moyenne = 10)
        printf("Moyenne juste suffisante\n");
    printf("Moyenne = %d\n", moyenne);
    return 0;
}
```

4. Liste les 5 erreurs en précisant la ligne concernée et la correction à apporter.
5. Après correction, quel format faut-il utiliser dans le dernier `printf` pour afficher `moyenne`?

#### EXERCICE 18 Type Bac — Recherche séquentielle dans un tableau

Le proviseur du lycée de Maroua veut vérifier rapidement si un matricule figure dans la liste des 10 élèves boursiers de l'établissement. On modélise la liste par un tableau de 10 entiers.

Partie A. On donne l'algorithme de la recherche séquentielle :

```
Algorithme RechercheSequentielle
Variables T : Tableau[1..10] de Entier
Variables i, x : Entier
Variable trouve : Booleen
Debut
    Pour i allant de 1 a 10 Faire
        Ecrire("Element ", i, " : ")
        Lire(T[i])
```

```
FinPour
Ecrire("Valeur a rechercher : ")
Lire(x)
trouve <- Faux
i <- 1
Tant que (i <= 10) et (trouve = Faux) Faire
    Si (T[i] = x) Alors
        trouve <- Vrai
    Sinon
        i <- i + 1
    FinSi
FinTantQue
Si (trouve = Vrai) Alors
    Ecrire("Valeur trouvee a la position ", i)
Sinon
    Ecrire("Valeur absente")
FinSi
Fin
```

1. Rappelle le principe de la recherche séquentielle dans un tableau.
2. En langage C, les indices d'un tableau de 10 éléments vont de 0 à 9. Réécris la condition de la boucle `Tant que` en conséquence.
3. Écris le programme C complet traduisant cet algorithme : déclaration du tableau, saisie des 10 éléments, saisie de la valeur `x`, recherche séquentielle (avec une boucle `while` et un booléen représenté par un entier), affichage du résultat.
4. On saisit le tableau : 15, 42, 7, 23, 56, 11, 90, 34, 8, 61. Donne l'affichage produit par le programme pour `x = 23`, puis pour `x = 100`.

#### Partie B.

5. Quel est le nombre maximal de comparaisons effectuées lors d'une recherche séquentielle dans un tableau de 10 éléments ? Justifie.
6. Propose une modification du programme pour qu'il affiche toutes les positions où la valeur `x` apparaît dans le tableau (au lieu de s'arrêter à la première).

## 12 CORRIGÉS DES EXERCICES

### 12.1 Corrigés des exercices

#### **CORRIGÉ** Exercice 1 — QCM : les bases du langage C

1. b) `printf("%d", n);`

Le spécificateur de format `%d` (ou `%i`) est utilisé pour afficher un entier (**int**). L'option a) affiche la lettre « n », l'option c) est une instruction de lecture (`scanf`), l'option d) est une écriture algorithmique (**Écrire**), pas du C.

2. b) `float moyenne;`

En C, le type pour les nombres réels (virgule flottante simple précision) est `float`. `int` est pour les entiers, `real` n'existe pas en C standard, et la syntaxe `float : moyenne;` est invalide (pas de deux-points en déclaration C).

3. c) `stdio.h`

**stdio.h** (Standard Input/Output) contient les prototypes de `printf` et `scanf`. `math.h` est pour les fonctions mathématiques, `stdlib.h` pour utilitaires généraux, `string.h` pour chaînes de caractères.

4. b) indiquer l'adresse mémoire de la variable `age`

L'opérateur `&` (adresse de) permet à `scanf` de modifier la variable en connaissant sa localisation en mémoire. Sans `&`, `scanf` recevrait la valeur de la variable (souvent du garbage) et écrirait à une adresse aléatoire → plantage ou corruption mémoire.

#### **CORRIGÉ** Exercice 2 — Vrai ou faux — justifier

1. Faux. En C, `=` est l'opérateur d'affectation (ex : `x = 5;`). Le test d'égalité s'écrit `==` (ex : `if (x == 5)`). Confondre les deux est une source majeure de bugs (ex : `if (x = 5)` affecte 5 à `x` et la condition est toujours vraie car non-nulle).
2. Vrai. Tout programme C exécutable doit posséder exactement une fonction `main` (point d'entrée). Elle retourne un `int` (code de retour au système d'exploitation, 0 = succès).
3. Faux. En C, les tableaux sont indicés à partir de 0. Pour `int T[10];`, les indices valides sont 0 à 9. `T[1]` est le deuxième élément. `T[10]` est hors limites (comportement indéfini / buffer overflow).
4. Vrai. La boucle `do { ... } while(condition);` est une boucle post-testée : le corps est exécuté une première fois avant l'évaluation de la condition.
5. Vrai. Le code source C est un simple fichier texte (extension `.c`). On peut l'écrire avec n'importe quel éditeur (Bloc-notes, Notepad++, Vim, VS Code). La transformation en

exécutable nécessite un compilateur (ex : GCC via Dev-C++ ou Code ::Blocks) et un éditeur de liens (linker).

### **CORRIGÉ** Exercice 3 — Définitions et vocabulaire

1. **Compilateur** : Programme qui traduit le code source (langage haut niveau, ex : C) en code machine (binaire) ou en code objet, en vérifiant la syntaxe. L'exécution se fait ensuite sans le compilateur.  
**IDE (Integrated Development Environment)** : Environnement de développement intégré qui regroupe au minimum un éditeur de code (coloration syntaxique, complétion), un compilateur, un éditeur de liens (linker) et un débogueur dans une seule interface graphique.  
**Enregistrement (struct)** : Type de données structuré défini par le programmeur qui regroupe sous un même nom plusieurs champs (membres) de types éventuellement différents (ex : un entier, un réel, une chaîne), contrairement au tableau qui ne contient que des éléments de même type.
2. Dev-C++ et Code ::Blocks (les deux cités au programme officiel MINESEC).
3. **Langage compilé** : Traduction complète du source en exécutable avant l'exécution (ex : C, C++, Pascal). Erreurs de syntaxe détectées à la compilation. Exécution rapide.  
**Langage interprété** : Traduction et exécution ligne par ligne (ou instruction par instruction) par un programme interprète au moment de l'exécution (ex : Python, JavaScript, BASIC). Plus lent, mais plus flexible (portabilité, exécution interactive).  
Le langage C est un langage compilé.
4. La directive `#include <stdio.h>` demande au préprocesseur d'insérer le contenu du fichier d'en-tête `stdio.h` (Standard Input/Output). Ce fichier contient les prototypes (déclarations) des fonctions d'entrée/sortie standard : `printf`, `scanf`, `puts`, `gets`, `fopen`, etc., ainsi que des macros (`NULL`, `EOF`) et types (`FILE`, `size_t`). Sans elle, le compilateur ne connaît pas ces fonctions → erreur « implicit declaration of function ».

### **CORRIGÉ** Exercice 4 — Correspondance algorithme → langage C

| Écriture algorithmique                    | Traduction en C                   |
|-------------------------------------------|-----------------------------------|
| Variable n : Entier                       | int n;                            |
| Lire(n)                                   | scanf("%d", &n);                  |
| Écrire("Bonjour")                         | printf("Bonjour");                |
| Si (n > 0) Alors ... FinSi                | if (n > 0) { ... }                |
| Pour i allant de 1 à 10 Faire ... FinPour | for (i = 1; i <= 10; i++) { ... } |
| Tant que (n < 100) Faire ... FinTantQue   | while (n < 100) { ... }           |

Remarques : En C, les blocs d'instructions sont délimités par des accolades { } (obligatoires si plusieurs instructions, recommandées toujours). L'indentation n'a pas de valeur syntaxique mais est indispensable pour la lisibilité. L'opérateur ++ incrémente de 1.

### **CORRIGÉ** Exercice 5 — Mon premier programme complet (Cybercafé)

```
#include <stdio.h>

int main()
{
    int heures;      /* nombre d'heures de connexion (entier) */
    int montant;     /* montant à payer en FCFA (entier) */
    const int TARIF = 300; /* tarif horaire constant */

    /* 1. Saisie */
    printf("Entrez le nombre d'heures de connexion : ");
    scanf("%d", &heures);

    /* 2. Calcul */
    montant = heures * TARIF;

    /* 3. Affichage du résultat */
    printf("Montant à payer : %d FCFA\n", montant);

    return 0;
}
```

Points clés : Inclusion de `stdio.h`, fonction `main` retournant `int`, déclaration des variables avant toute instruction (standard C90 souvent exigé au lycée, bien que C99 permette déclaration n'importe où), utilisation de `&` dans `scanf`, `%d` pour entiers, `\n` pour retour à la ligne.

### **CORRIGÉ** Exercice 6 — Traduction de structures de contrôle

1. Programme C complet (version simple) :

```
#include <stdio.h>

int main()
{
    int n;

    printf("Entrez un entier : ");
    scanf("%d", &n);

    if (n % 2 == 0)
        printf("Pair\n");
    else
        printf("Impair\n");

    return 0;
}
```

% est l'opérateur modulo (reste de la division entière). `n % 2 == 0` teste la parité.

2. Version avec boucle `for` pour 5 entiers :

```
#include <stdio.h>

int main()
{
    int n, i;

    for (i = 1; i <= 5; i++)
    {
        printf("Entier %d : ", i);
        scanf("%d", &n);

        if (n % 2 == 0)
            printf("%d est Pair\n", n);
        else
            printf("%d est Impair\n", n);
    }

    return 0;
}
```

La variable de boucle `i` sert à numéroter les saisies. Les accolades `{ }` délimitent le corps de la boucle `for`.

### **CORRIGÉ** Exercice 7 — Tableau de notes

```
#include <stdio.h>

int main()
{
    float notes[8]; /* tableau de 8 réels : indices 0 à 7 */
    float somme = 0.0f;
    float moyenne;
```

```

float max;
int i;

/* 1. Saisie des 8 notes */
for (i = 0; i < 8; i++)
{
    printf("Note %d : ", i + 1); /* affichage 1..8 pour l'utilisateur */
    scanf("%f", &notes[i]);
}

/* 2. Calcul de la somme et recherche du maximum */
max = notes[0]; /* initialisation avec le premier élément */
for (i = 0; i < 8; i++)
{
    somme = somme + notes[i];
    if (notes[i] > max)
        max = notes[i];
}

/* 3. Calcul de la moyenne */
moyenne = somme / 8.0f;

/* 4. Affichage des résultats */
printf("Somme des notes : %.2f\n", somme);
printf("Moyenne : %.2f\n", moyenne);
printf("Plus grande note : %.2f\n", max);

return 0;
}

```

#### Points de vigilance :

- Indices du tableau : 0 à 7 (taille 8). La boucle utilise `i < 8`.
- Initialisation de `max` avec `notes[0]` (suppose au moins un élément, ce qui est le cas ici).
- Format `%.2f` pour afficher deux décimales (lisibilité).
- `8.0f` force la division flottante (bien que `somme` soit `float`, c'est une bonne habitude).

#### **CORRIGÉ** Exercice 8 — Trace d'exécution

##### 1. Table d'exécution :

| Étape          | i | s | Action                                        |
|----------------|---|---|-----------------------------------------------|
| Initialisation | 1 | 0 | <code>i=1; s=0;</code>                        |
| Itération 1    | 1 | 1 | <code>s=s+i (0+1); i=i+2 (3);</code> Afficher |
| Itération 2    | 3 | 4 | <code>s=s+i (1+3); i=i+2 (5);</code> Afficher |
| Itération 3    | 5 | 9 | <code>s=s+i (4+5); i=i+2 (7);</code> Afficher |
| Test sortie    | 7 | 9 | <code>i&lt;=5 ? Faux (7&gt;5) → Fin</code>    |

## 2. Affichage exact à l'écran :

```
i = 1, s = 1
i = 3, s = 4
i = 5, s = 9
Fin : s = 9
```

3. Si `i = i;` (au lieu de `i = i + 2;`) : La variable `i` ne change plus de valeur (reste 1). La condition `i <= 5` reste toujours vraie. La boucle `while` devient infinie (bouclage sans fin). Le programme afficherait indéfiniment `i = 1, s = 1` (puis `s` augmenterait : 1, 2, 3... mais `i` bloqué à 1). Il faut tuer le processus (Ctrl+C ou fermer la console).

### **CORRIGÉ** Exercice 9 — Type Bac — Moyenne générale d'un élève

#### 1. Rôle des directives :

- `#include <stdio.h>` : Indispensable. Fournit les prototypes de `printf` et `scanf` utilisés pour l'affichage et la saisie.
- `#include <math.h>` : Fournit les fonctions mathématiques (`sqrt`, `pow`, `cos`, etc.). Non indispensable ici car l'algorithme n'utilise aucune fonction mathématique avancée (seulement `+`, `*`, `/`).

#### 2. Déclaration des variables en C :

```
int i, coef, sommeCoef;
float note, sommeNotes, moyenne;
```

`int` pour les entiers (compteur, coefficient, somme des coefficients). `float` pour les réels (note, somme pondérée, moyenne).

#### 3. Traduction de la boucle Pour (lignes 6 à 13 de l'algo) :

```
sommeNotes = 0.0f;
sommeCoef = 0;
for (i = 1; i <= 5; i++)
{
    printf("Note numero %d : ", i);
    scanf("%f", &note);
    printf("Coefficient : ");
    scanf("%d", &coef);
    sommeNotes = sommeNotes + note * coef;
    sommeCoef = sommeCoef + coef;
}
```

#### 4. Programme C complet :

```
#include <stdio.h>

int main()
{
    int i, coef, sommeCoef;
    float note, sommeNotes, moyenne;
```

```
sommeNotes = 0.0f;
sommeCoef = 0;

for (i = 1; i <= 5; i++)
{
    printf("Note numero %d : ", i);
    scanf("%f", &note);
    printf("Coefficient : ");
    scanf("%d", &coef);
    sommeNotes = sommeNotes + note * coef;
    sommeCoef = sommeCoef + coef;
}

moyenne = sommeNotes / sommeCoef;
printf("Moyenne generale : %.2f\n", moyenne);

if (moyenne >= 10.0f)
    printf("Admis(e)\n");
else
    printf("Ajourné(e)\n");

return 0;
}
```

5. Calcul avec les données fournies :

Notes ( $n_i$ ) : 12, 8, 14, 10, 9

Coefficients ( $c_i$ ) : 3, 2, 4, 1, 2

$$\begin{aligned}\text{Somme pondérée } \sum n_i c_i &= 12 \times 3 + 8 \times 2 + 14 \times 4 + 10 \times 1 + 9 \times 2 \\ &= 36 + 16 + 56 + 10 + 18 = 136\end{aligned}$$

$$\text{Somme des coefficients } \sum c_i = 3 + 2 + 4 + 1 + 2 = 12$$

$$\text{Moyenne} = \frac{136}{12} = 11,33 \text{ (arrondi à 2 décimales)}$$

Décision :  $11,33 \geq 10 \Rightarrow \text{Admis(e)}$

#### **CORRIGÉ** Exercice 10 — Type Bac — Complétion et débogage (Struct Produit)

##### Partie A : Complétion

1. Type pour `prix` : `float` (ou `double`).

Justification : Le commentaire indique « prix unitaire en FCFA (reel) ». Bien que les prix en FCFA soient souvent des entiers, l'énoncé impose un type réel pour permettre des valeurs décimales (ex : prix au kg, ou devise avec centimes si adaptation). `float` suffit pour la précision demandée (7 chiffres significatifs).

2. Code complété :

```

#include <stdio.h>

struct Produit
{
    char nom[30];
    float prix;      /* prix unitaire en FCFA (reel) */
    int quantite;
};

int main()
{
    struct Produit p1, p2, p3;
    struct Produit maxProd; /* évite conflit avec macro max */
    int i;

    /* saisie des 3 produits (code suppose écrit) */

    maxProd = p1;
    if (p2.prix > maxProd.prix) /* 1er pointillé : opérateur > */
        maxProd = p2;
    if (p3.prix > maxProd.prix) /* 2e, 3e, 4e pointillés : p3.prix > maxProd.prix */
        maxProd = p3;

    printf("Produit le plus cher : %s a %.2f FCFA\n", /* %f ou %.2f pour float */
        maxProd.nom, maxProd.prix);
    return 0;
}

```

### 3. Structure de données : C'est une structure (mot-clé struct). Définition complète :

```

struct Produit
{
    char nom[30];
    float prix;
    int quantite;
};

```

Elle permet de regrouper des champs de types hétérogènes (char[], float, int) sous un même nom de type (struct Produit).

#### Partie B : Débogage (5 erreurs)

##### 4. Liste des 5 erreurs :

- (a) Ligne 8 (boucle for) : `for (i = 1; i <= 3; i++)`. Les indices d'un tableau `notes[3]` vont de 0 à 2. Ici `i` prend les valeurs 1, 2, 3 → `notes[3]` est hors limites (dépassement de capacité / buffer overflow). Correction : `for (i = 0; i < 3; i++)` (et adapter l'affichage `printf("Note %d : ", i+1);`).
- (b) Ligne 11 (scanf) : `scanf("%f", notes[i]);`. `scanf` attend l'adresse de la variable pour la modifier. Il manque l'opérateur `&`. Correction : `scanf("%f", &notes[i]);`.
- (c) Ligne 12 (affectation) : `somme = somme + notes[i]`. Il manque le point-virgule ; en fin d'instruction. Correction : `somme = somme + notes[i];`.
- (d) Ligne 15 (if) : `if (moyenne = 10)`. C'est une affectation (=) et non un test d'égalité.

(==). La condition sera toujours vraie (valeur affectée 10, non-nulle) et `moyenne` sera modifiée. Correction : `if (moyenne == 10)` (ou mieux, test sur intervalle pour les flottants : `fabs(moyenne - 10) < 0.001`).

(e) Ligne 17 (`printf` final) : `printf("Moyenne = %d\n", moyenne);`. Le spécificateur `%d` attend un `int`, or `moyenne` est un `float`. Comportement indéfini (affichage aberrant). Correction : `printf("Moyenne = %.2f\n", moyenne);` (ou `%f`).

5. Après correction, le format correct pour afficher `moyenne` (type `float`) est `%f` (ou `%.2f` pour 2 décimales).

### **CORRIGÉ** Exercice 11 — Type Bac — Recherche séquentielle dans un tableau

#### Partie A

1. Principe : La recherche séquentielle (ou linéaire) parcourt le tableau élément par élément, du premier au dernier, en comparant chaque élément à la valeur recherchée `x`. Elle s'arrête dès que l'élément est trouvé (succès) ou si la fin du tableau est atteinte sans l'avoir trouvé (échec). C'est la méthode la plus simple, mais de complexité  $O(n)$ .
2. Condition de la boucle `Tant` que adaptée aux indices C (0 à 9) :  
L'algorithme original utilise `i` commençant à 1 et condition `i <= 10`. En C, `i` commence à 0 et la condition devient :

```
while (i < 10 && trouve == 0)
```

(avec `i` initialisé à 0 et `trouve` à 0 pour faux).

3. Programme C complet :

```
#include <stdio.h>

int main()
{
    int T[10];    /* tableau de 10 entiers : indices 0..9 */
    int i, x;
    int trouve = 0; /* booléen : 0 = Faux, 1 = Vrai */

    /* Saisie des 10 éléments */
    for (i = 0; i < 10; i++)
    {
        printf("Element %d : ", i + 1);
        scanf("%d", &T[i]);
    }

    /* Saisie de la valeur à rechercher */
    printf("Valeur a rechercher : ");
    scanf("%d", &x);

    /* Recherche séquentielle */
    trouve = 0;
```

```
i = 0;
while (i < 10 && trouve == 0)
{
    if (T[i] == x)
        trouve = 1;
    else
        i = i + 1;
}

/* Affichage du résultat */
if (trouve == 1)
    printf("Valeur trouvee a la position %d\n", i + 1); /* position humaine 1..10 */
else
    printf("Valeur absente\n");

return 0;
}
```

4. Affichages pour le tableau : 15, 42, 7, 23, 56, 11, 90, 34, 8, 61 (indices 0 à 9).

- $x = 23$  :  $T[3] == 23$ . La boucle s'arrête à  $i = 3$ .  $trouve = 1$ .  
Affichage : Valeur trouvee a la position 4
- $x = 100$  : Aucune correspondance. La boucle va jusqu'à  $i = 10$  (condition  $i < 10$  devient fausse).  $trouve = 0$ .  
Affichage : Valeur absente

### Partie B

5. Nombre maximal de comparaisons : 10.

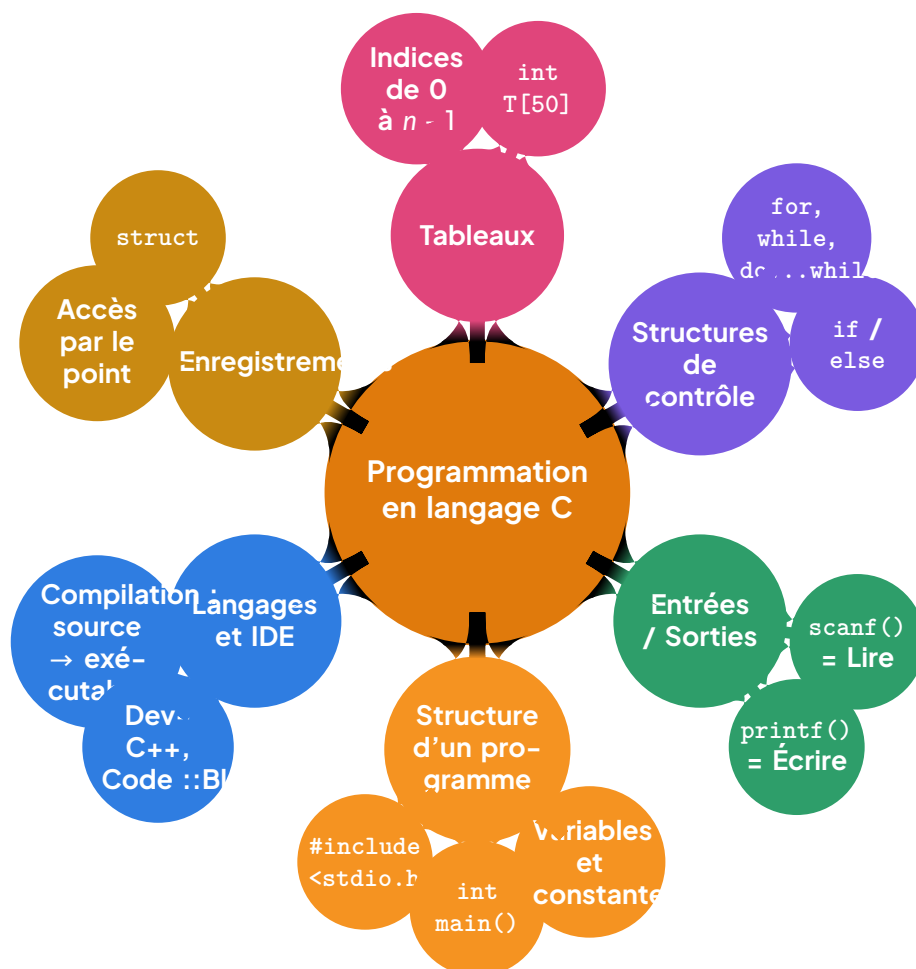
Justification : Dans le pire des cas, la valeur recherchée  $x$  est absente du tableau (ou se trouve à la dernière position). La boucle `while` compare  $x$  avec chacun des 10 éléments ( $T[0]$  à  $T[9]$ ) avant de s'arrêter. Il y a donc exactement 10 comparaisons  $T[i] == x$ .

6. Modification pour afficher toutes les positions :

On remplace la boucle `while` (qui s'arrête à la première occurrence) par une boucle `for` qui parcourt tout le tableau, et on affiche à chaque correspondance. On utilise un drapeau `trouve` pour savoir s'il y a eu au moins une occurrence.

```
trouve = 0;
for (i = 0; i < 10; i++)
{
    if (T[i] == x)
    {
        printf("Valeur trouvee a la position %d\n", i + 1);
        trouve = 1;
    }
}
if (trouve == 0)
    printf("Valeur absente\n");
```

## 13 FICHE BILAN



Carte mentale de la leçon : les six piliers de la programmation en langage C.

### À RETENIR L'essentiel de la leçon

#### 1. Définitions clés

- **Langage de programmation** : langage formel permettant d'écrire des programmes exécutables par l'ordinateur. Le **C** est un langage compilé de niveau intermédiaire.
- **Compilation** : traduction complète du programme source (fichier .c) en programme exécutable par un logiciel appelé **compilateur**.
- **IDE** (environnement de développement intégré) : logiciel réunissant éditeur de code, compilateur et débogueur. Exemples : Dev-C++, Code ::Blocks.
- **Bibliothèque** : fichier d'en-tête (stdio.h, math.h) contenant des fonctions prêtes à l'emploi.

l'emploi, inclus par `#include`.

- **Variable** : zone mémoire nommée et typée dont la valeur peut changer ; **constante** : valeur fixée à la déclaration (`const` ou `#define`).
- **Tableau** : ensemble de cases de même type accessibles par indice ; **struct** : enregistrement regroupant des champs de types différents.

#### PROPRIÉTÉ Syntaxes C de référence

| Notion                  | Syntaxe C                                                                    |
|-------------------------|------------------------------------------------------------------------------|
| Squelette minimal       | <code>#include &lt;stdio.h&gt; puis int main(){ ... return 0; }</code>       |
| Déclarations            | <code>int a; float moy; char c; const int N = 50;</code>                     |
| Écrire                  | <code>printf("Moyenne = %f\n", moy);</code>                                  |
| Lire                    | <code>scanf("%d", &amp;a);</code> (ne jamais oublier le <code>&amp;</code> ) |
| Formats                 | <code>%d (int), %f (float), %c (char), %s (chaîne)</code>                    |
| Condition               | <code>if (cond) { ... } else { ... }</code>                                  |
| Boucle bornée           | <code>for (i = 0; i &lt; n; i++) { ... }</code>                              |
| Boucles conditionnelles | <code>while (cond) { ... }; do { ... } while (cond);</code>                  |
| Tableau                 | <code>int T[50]; accès : T[i], indices de 0 à n - 1</code>                   |
| Enregistrement          | <code>struct Eleve { char nom[30]; float moy; }; puis e.moy</code>           |
| Fonctions maths         | <code>sqrt(x), pow(x, y), fabs(x)</code> (avec <code>math.h</code> )         |

#### MÉTHODE Traduire un algorithme en C

1. Inclure les bibliothèques nécessaires (`#include <stdio.h>`, `#include <math.h>` si besoin).
2. Ouvrir la fonction principale : `int main() { .`
3. Déclarer toutes les variables et constantes au début du bloc (avant toute instruction).
4. Remplacer Lire par `scanf` (avec `&` pour les variables simples) et Écrire par `printf` (formats `%d, %f, %c, %s`).
5. Traduire les structures de contrôle : Si...Sinon  $\rightarrow$  `if...else`, Pour  $\rightarrow$  `for`, Tant que  $\rightarrow$  `while`, Faire...Tant que  $\rightarrow$  `do...while`.
6. Fermer `main` par `return 0; }.`

7. Compiler, corriger les erreurs de syntaxe, puis exécuter et tester.

#### MÉTHODE Algorithme de recherche séquentielle

1. Déclarer le tableau `T`, sa taille effective `n`, la valeur cherchée `x`, un indice `i` et un booléen `trouve` (ou un entier `pos = -1`).
2. Parcourir le tableau avec `for (i = 0; i < n; i++)`.
3. À chaque itération, tester `if (T[i] == x)`.
4. Si vrai : affecter `pos = i` (ou `trouve = 1`) et sortir de la boucle avec `break`;
5. Après la boucle : si `pos != -1` (ou `trouve`), afficher la position ; sinon afficher « Valeur non trouvée ».

#### MÉTHODE Choisir la structure de contrôle adaptée

1. Nombre d'itérations connu à l'avance (ex. parcourir un tableau) → `for`.
2. Condition d'arrêt testée avant la première exécution (ex. saisie valide, lecture fichier) → `while`.
3. Au moins une exécution garantie (ex. menu interactif, saisie forcée) → `do...while`.

#### À RETENIR Checklist avant le Bac

- ☐ Je sais citer deux IDE (Dev-C++, Code ::Blocks) et expliquer le rôle du compilateur.
- ☐ J'écris de mémoire le squelette complet d'un programme C (`#include, main, return 0;`).
- ☐ Je connais les types de base : `int, float, char`, et le mot-clé `const`.
- ☐ Je maîtrise les formats `%d, %f, %c, %s` dans `printf` et `scanf`.
- ☐ Je n'oublie jamais le `&` devant la variable dans `scanf` (sauf pour les chaînes de caractères / tableaux).
- ☐ Je traduis correctement Si...Sinon, Pour, Tant que, Faire...Tant que en `if, for, while, do...while`.
- ☐ Je sais qu'un tableau de `n` cases est indicé de 0 à `n - 1` (premier élément `T[0]`).
- ☐ Je sais déclarer une `struct` et accéder à un champ avec le point (`variable.champ`).

- ☐ Je termine chaque instruction par un point-virgule ; et chaque bloc d'instructions par des accolades { }.
- ☐ Je sais écrire et commenter la recherche séquentielle dans un tableau.
- ☐ Je pense à inclure `math.h` dès que j'utilise `sqrt`, `pow`, `fabs`, etc.
- ☐ Je relis mon programme ligne par ligne avant de le rendre (test mental d'exécution).

**Signé • OnBuch+**Ludovic A.  
CONFORME AU PROGRAMME OFFICIEL MINESEC**OnBuch+**